# CS151 Intro to Data Structures

Trees

# Announcements

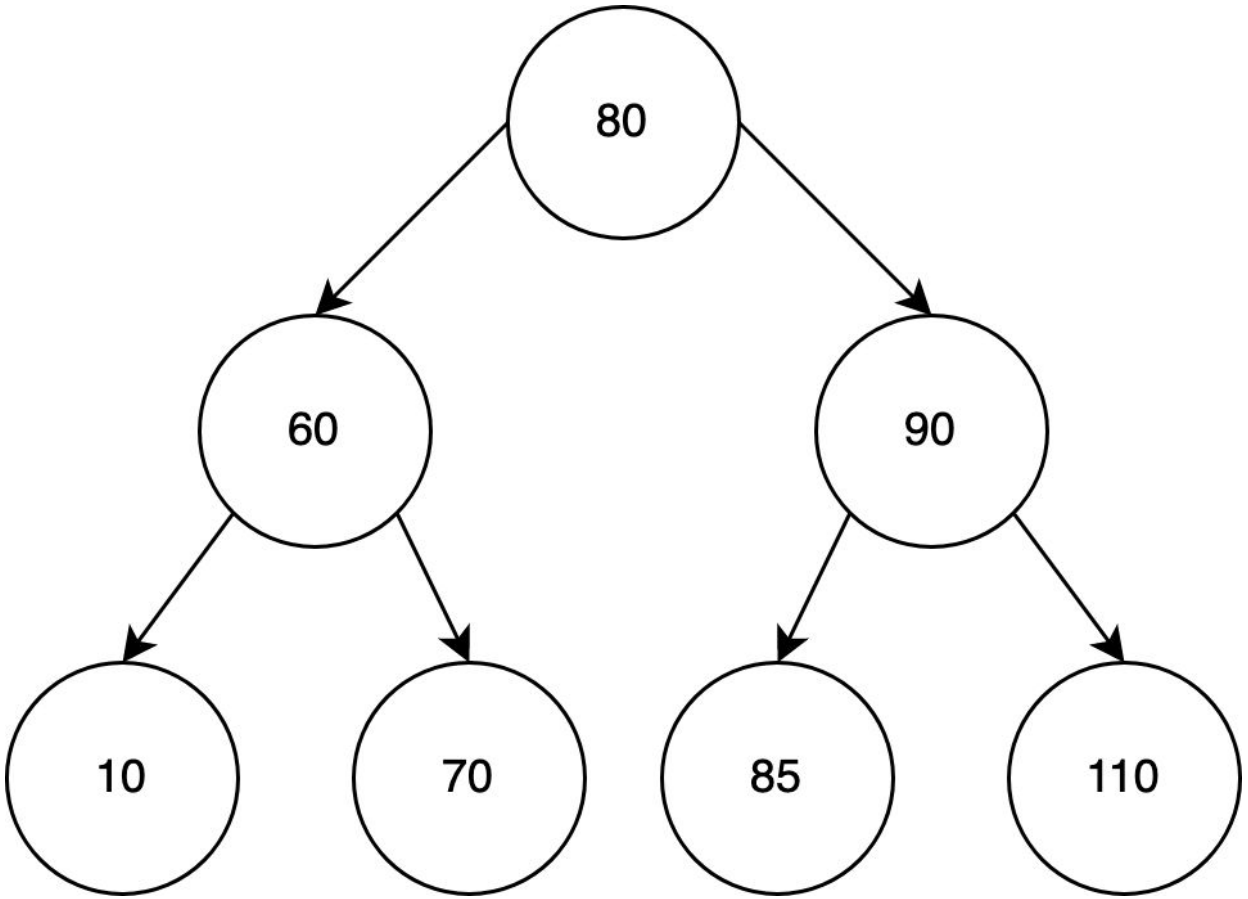
- Lab5 and HW3 due today
- Lab6 and HW4 due March 16th
- Practice exam questions will be posted on piazza

# Outline

- Trees:
  - Binary Trees
  - Binary Search Trees
    - Inserting
    - Searching

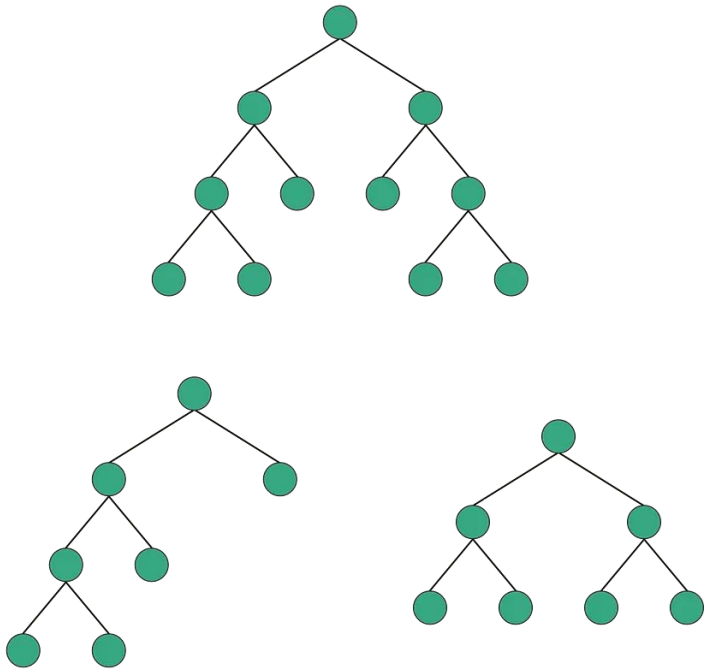
# Trees: Nodes

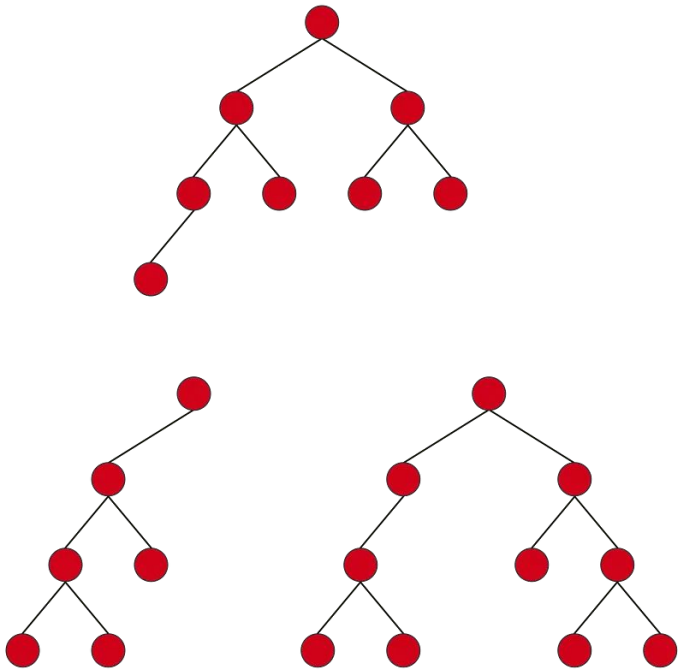
```
class Node {
    int key;
    Node left;
    Node right;

    public Node(int item) {
        key = item;
        left = null;
        right = null;
    }
}
```



# Types of Binary Trees

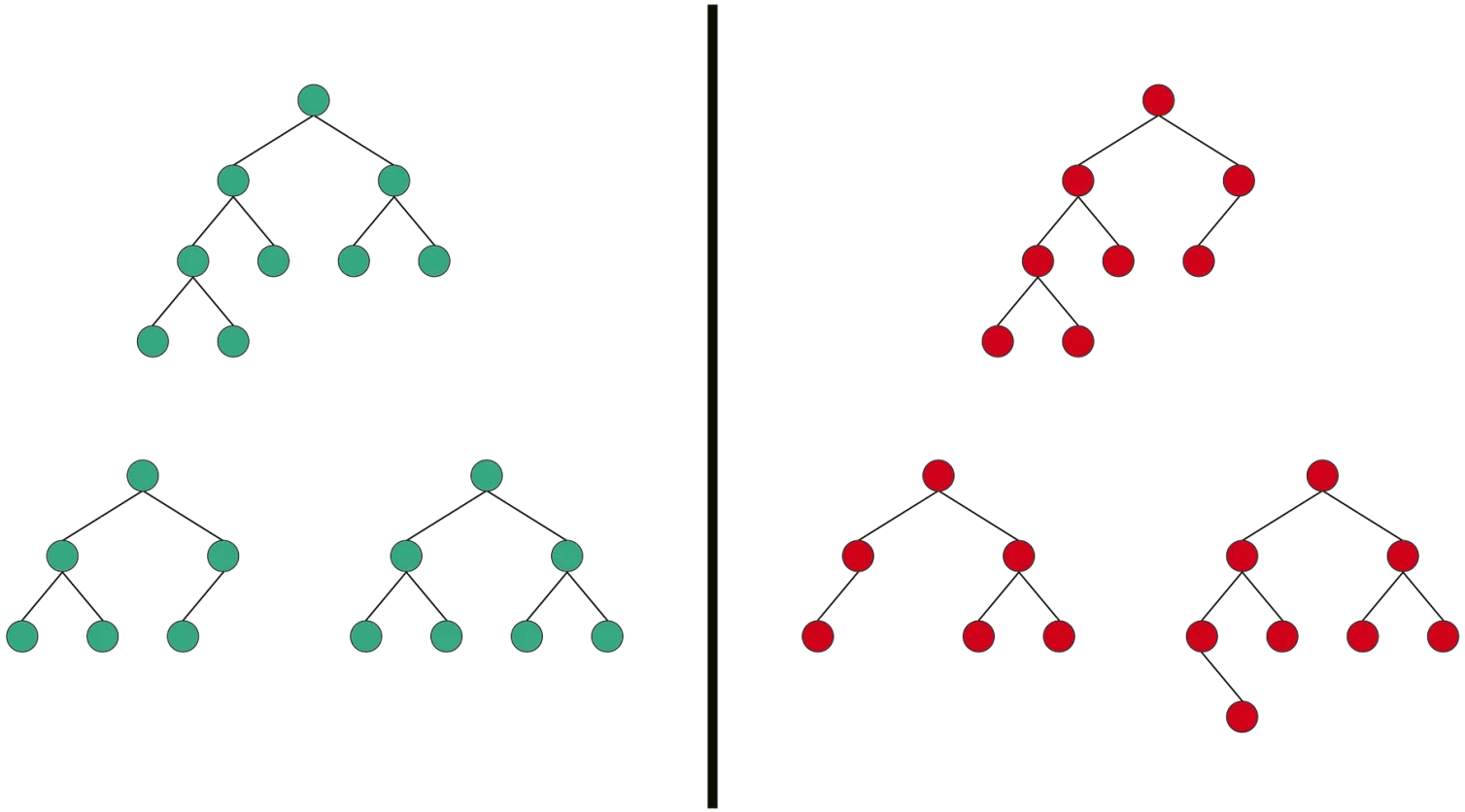
A binary tree is **full** (proper) if every node has 0 or 2 children





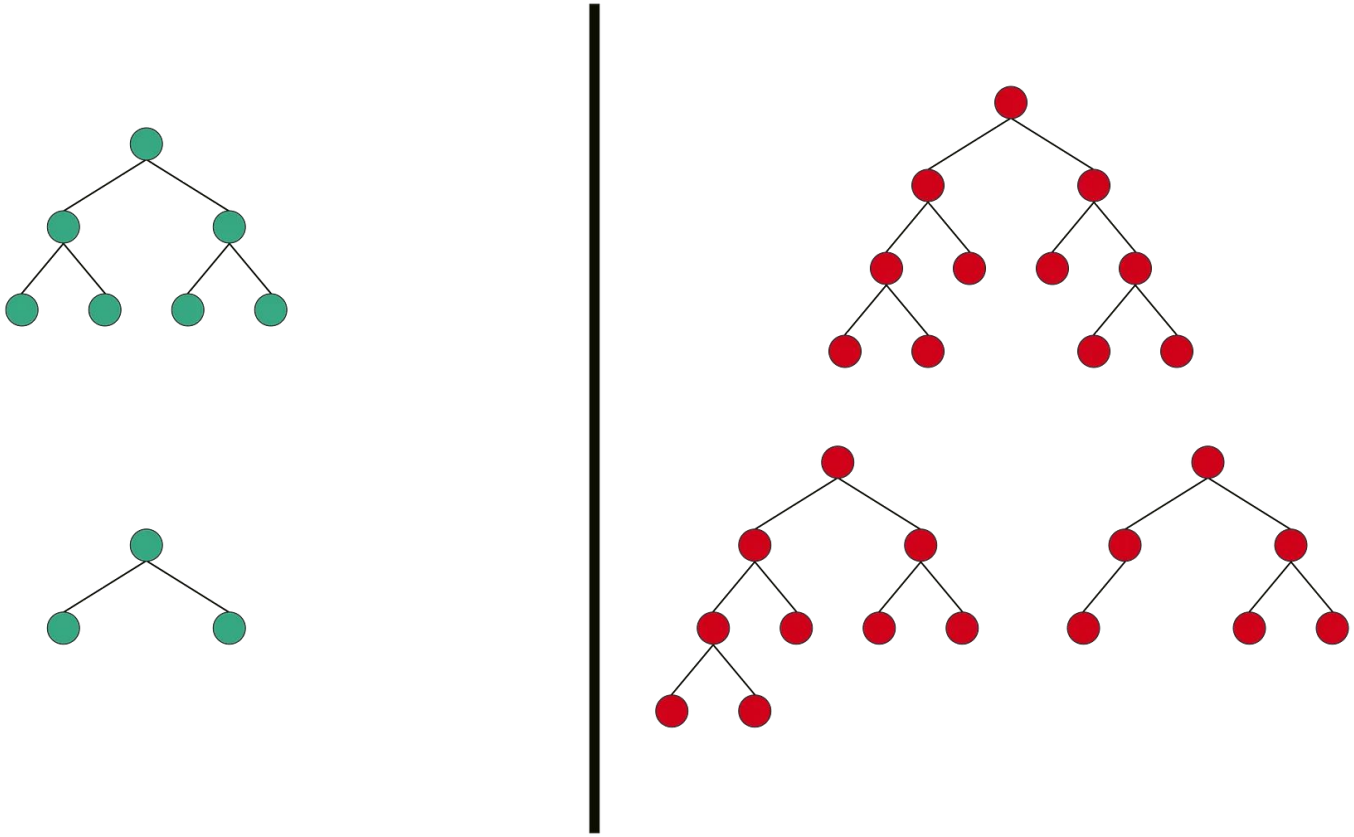
# Types of Binary Trees

A binary tree is **complete** if all levels are completely filled (zero or two children) *except possibly the last level* and all the nodes are as left side as possible



# Types of Binary Trees

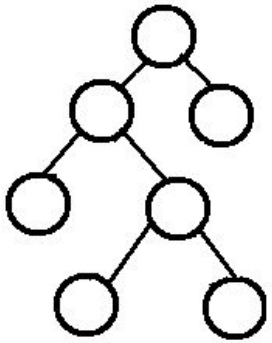
A binary tree is **perfect** if all internal (non-leaf) nodes have 2 children and all the leaf nodes are at the same depth or same level.



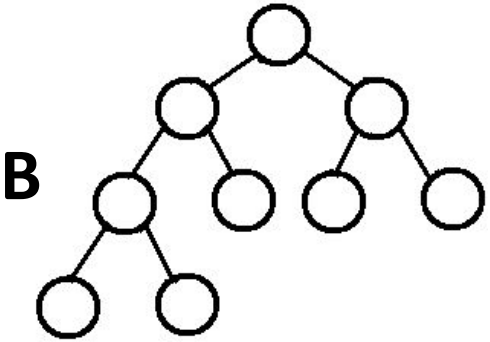
# Warmup: Types of Binary Trees

For each binary tree below, label it as full, complete, perfect, or any combination of these. A tree may satisfy more than one property, exactly one, or none.

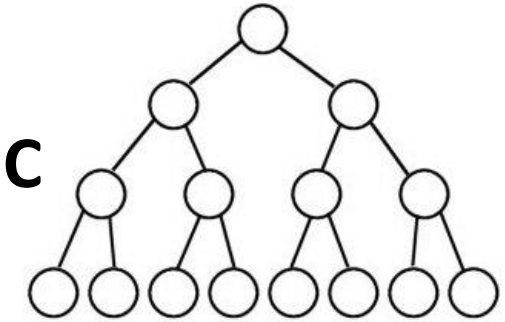
A



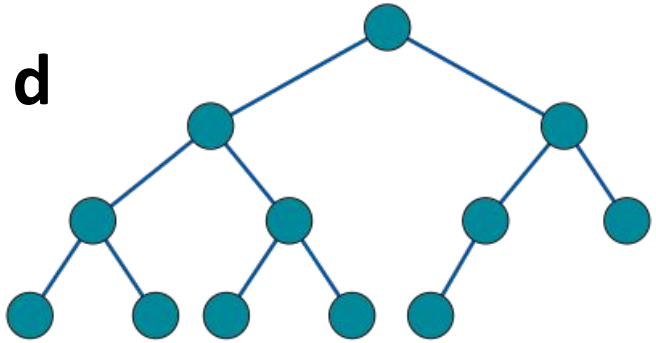
B



C



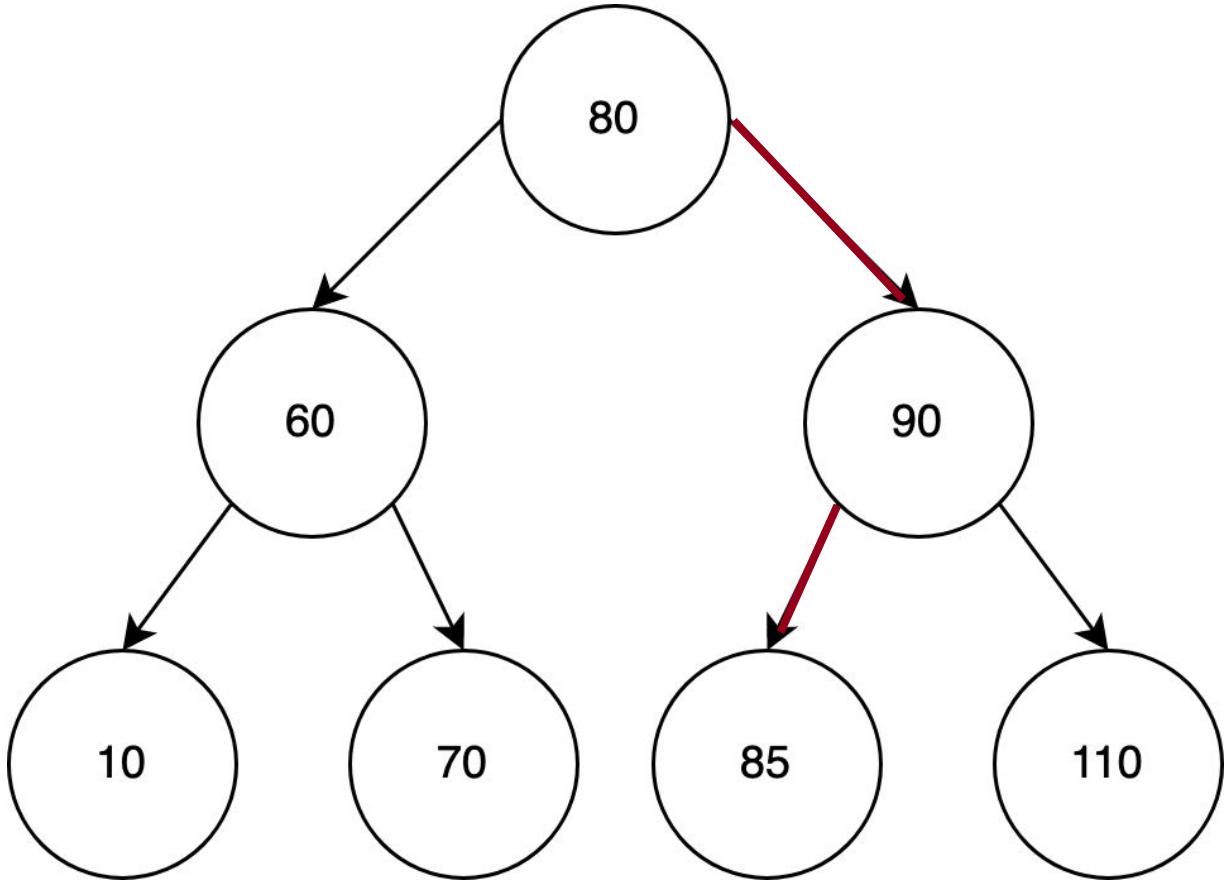
d



# Binary Trees: Height

## Height of a tree:

Maximum number of edges from a leaf node to the root




## Height? 2

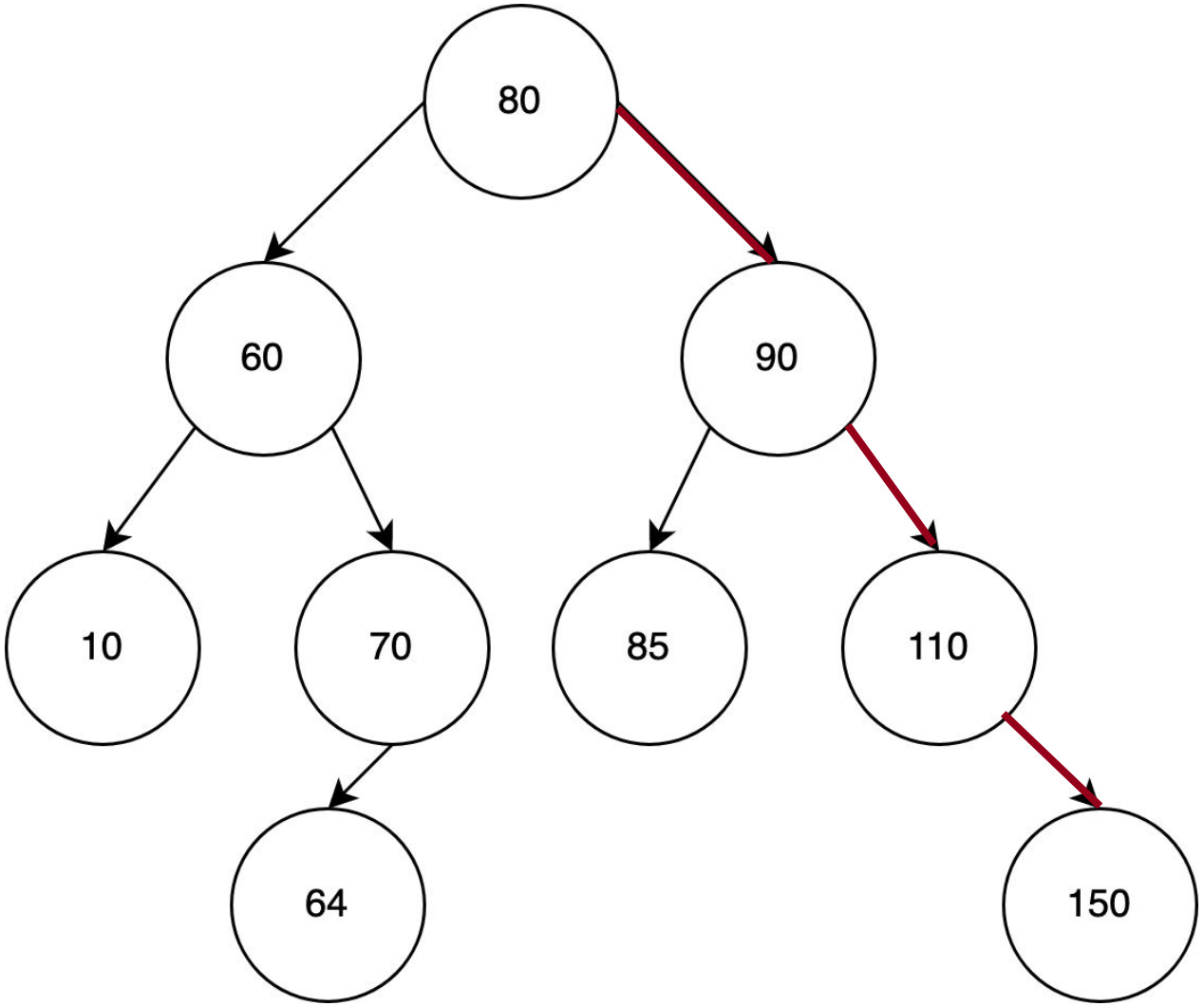
$\log_2(7) \approx 2$

# Tree Review

Height? 3

$\log_2(9) \approx 3$

Height of a binary tree is roughly log(n) where n is number of nodes

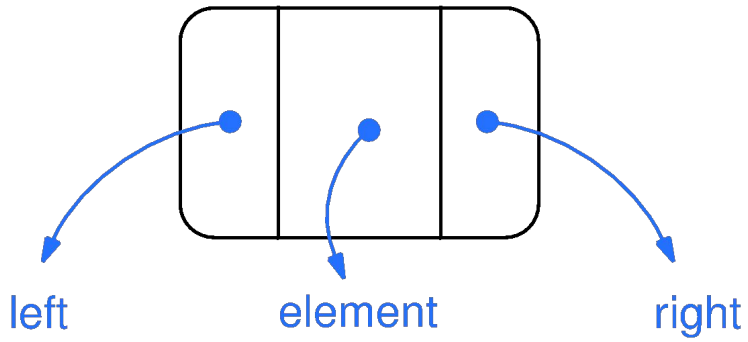


# Binary Tree Interface

```
public interface BinaryTree<E extends
Comparable<E>> {
  int size();
  boolean isEmpty();
  void insert(E element);
  boolean contains(E element);
  ...
}
```

# Node Implementation

```
public class Node<E> {
  private E element;
  private Node<E> left;
  private Node<E> right;
  //constructors, getters, setters
  }
}
```



# Class

```
public class LinkedBinaryTree<E extends
Comparable<E>> implements BinaryTree<E> {
  // what instance variables?
  // nested Node class


}
```

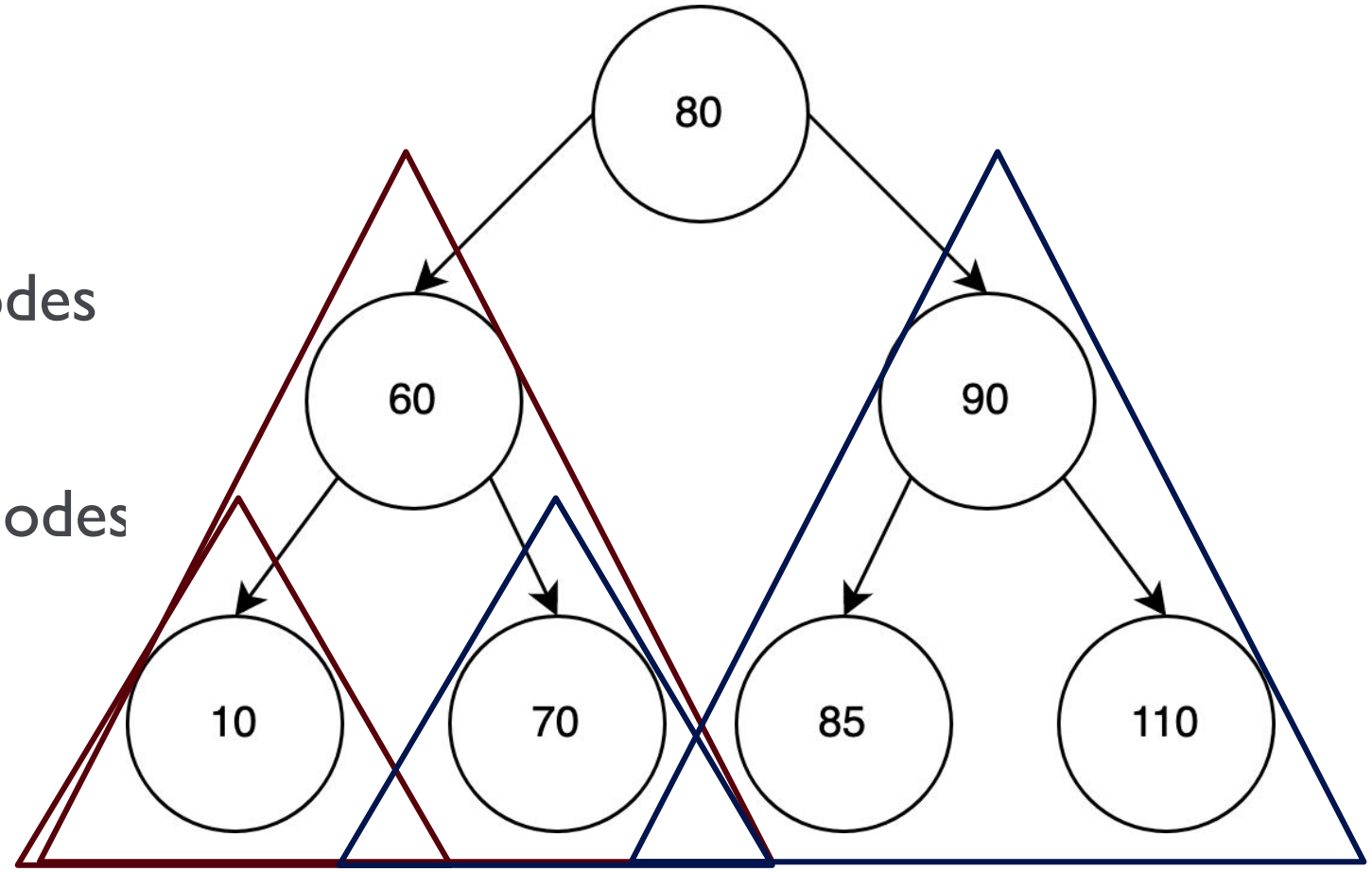
# Binary Search Trees

# Binary Search Trees

## Definition:

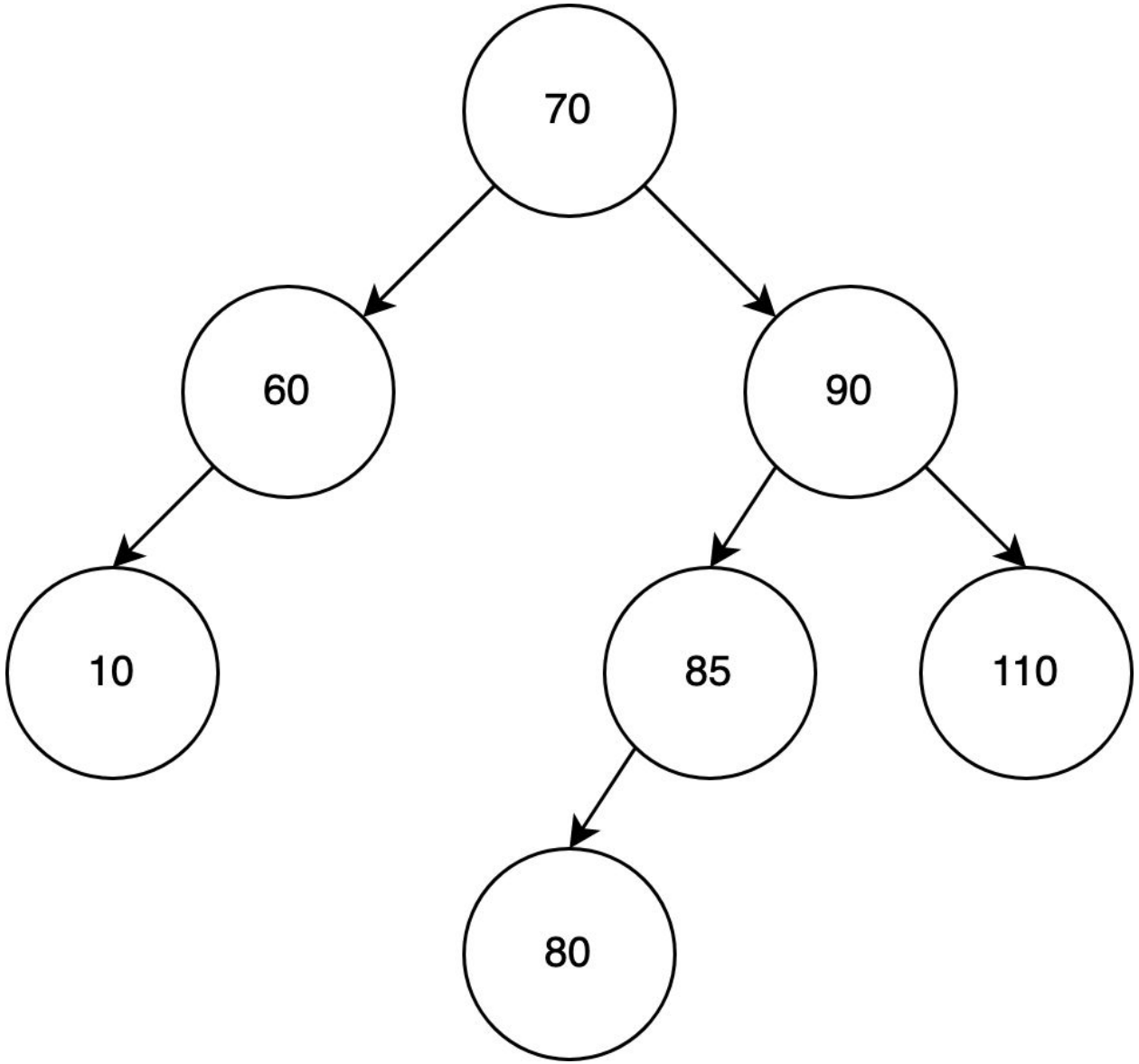
At **each node** with value **k**

- Left subtree contains only nodes with value **lesser** than **k**
- Right subtree contains only nodes with value **greater** than **k**
- Both subtrees are a **binary search tree**



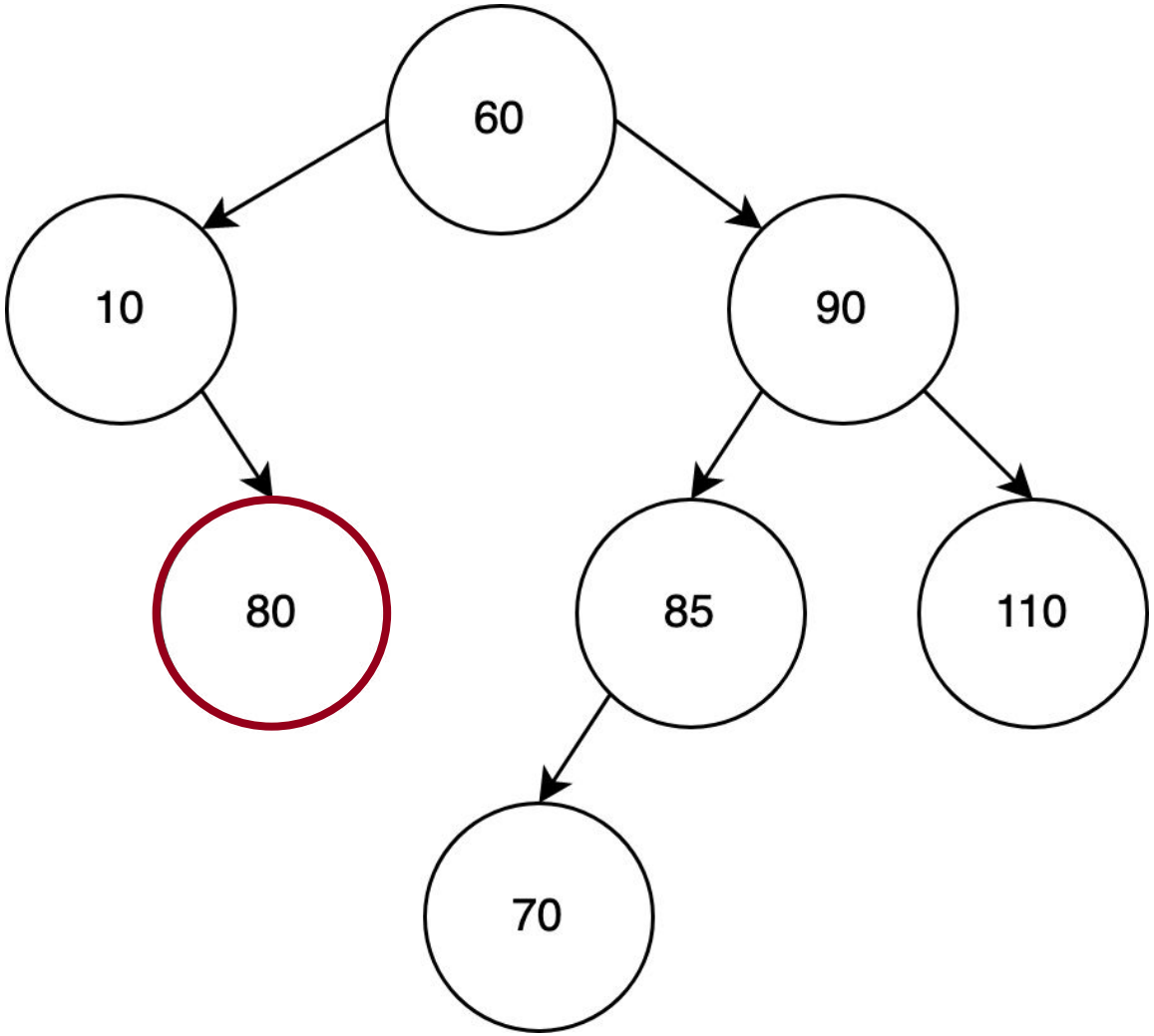
# Exercise One: Binary Search Trees

Is this a binary search tree?



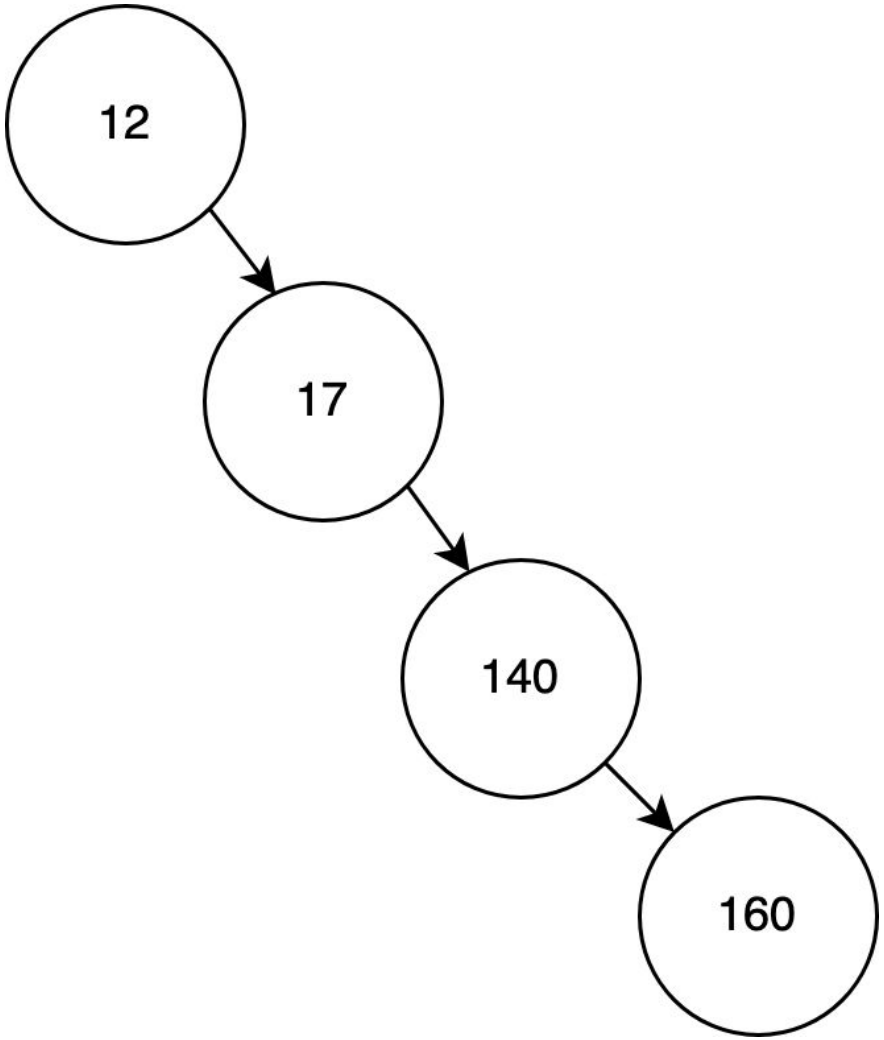
# Exercise One: Binary Search Trees

Is this a binary search tree?



# Exercise One: Binary Search Trees

Is this a binary search tree?



# Today's Lecture

1. Binary Search Trees
2. **Search**
3. Insertion
4. Removal
5. Summary

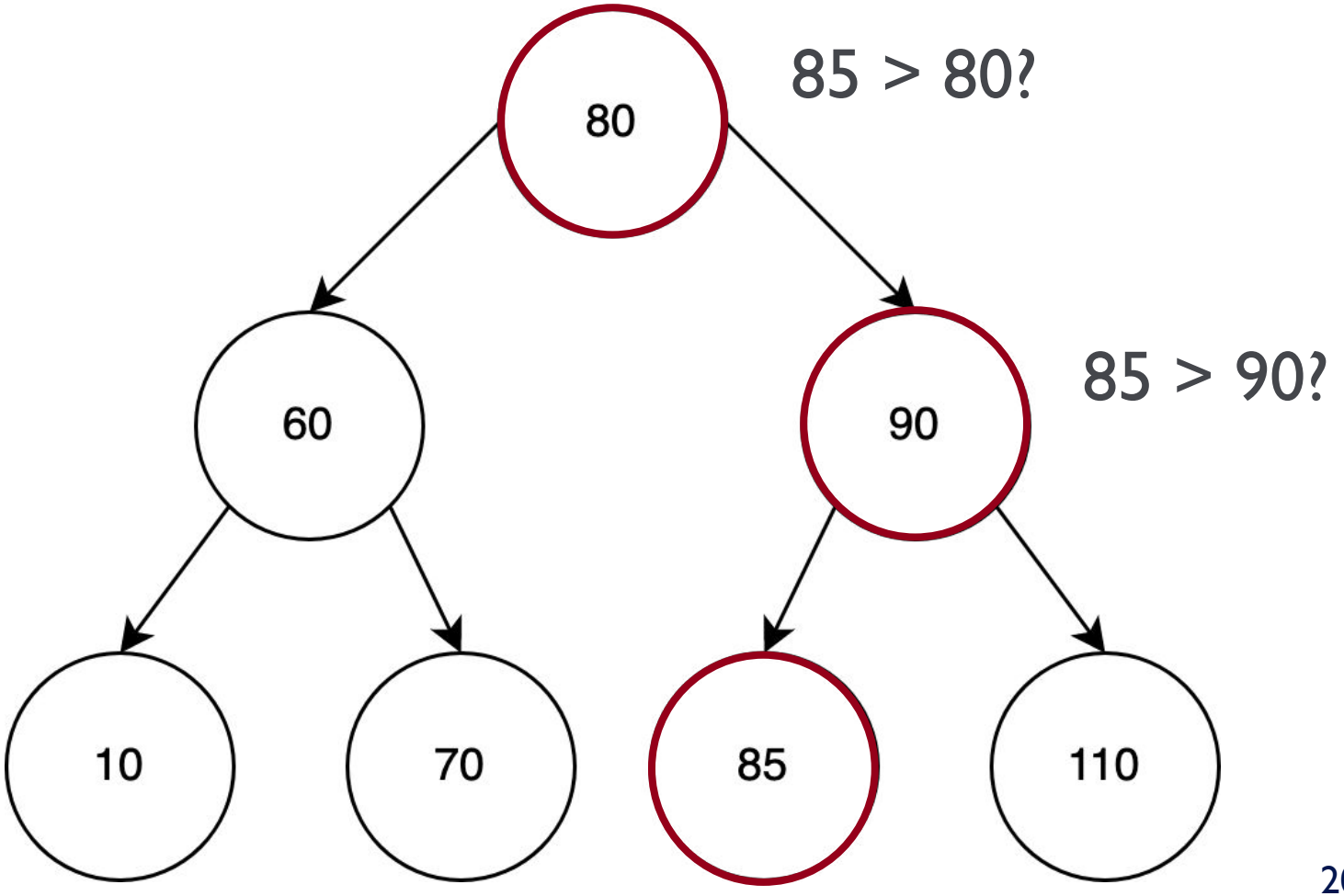
# Binary Search Trees: Efficient Search

Goal: Report if a value exists in the tree

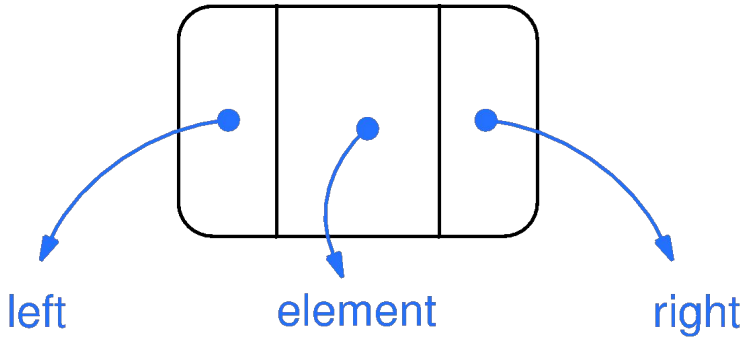
**Target**: 85

if **target** > **k**:
    Move right
else:
    Move Left

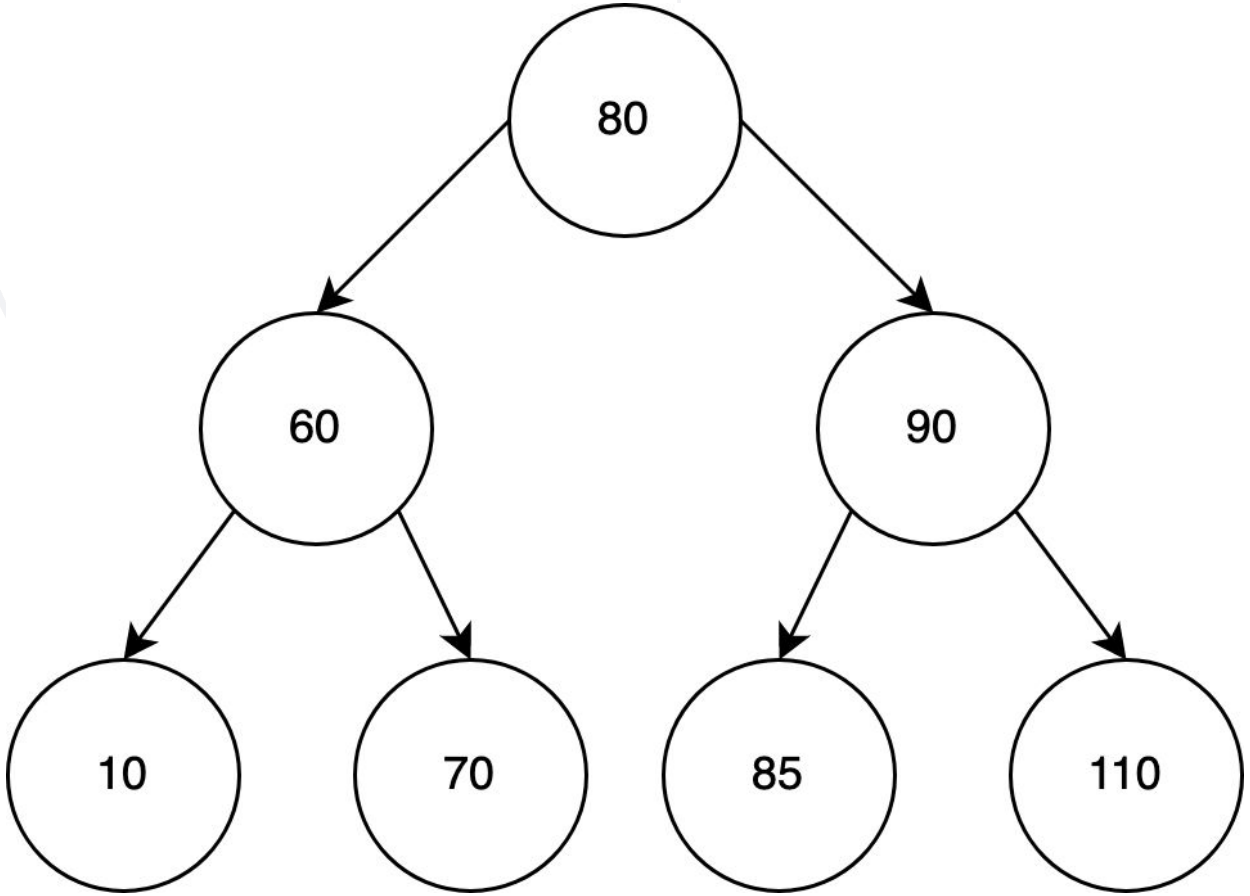
Complexity?
O(log n)



# BSTs: Search Implementation



# BSTs: Search Implementation




search(Node(80), 85)

search(Node(90), 85)

search(Node(85), 85)

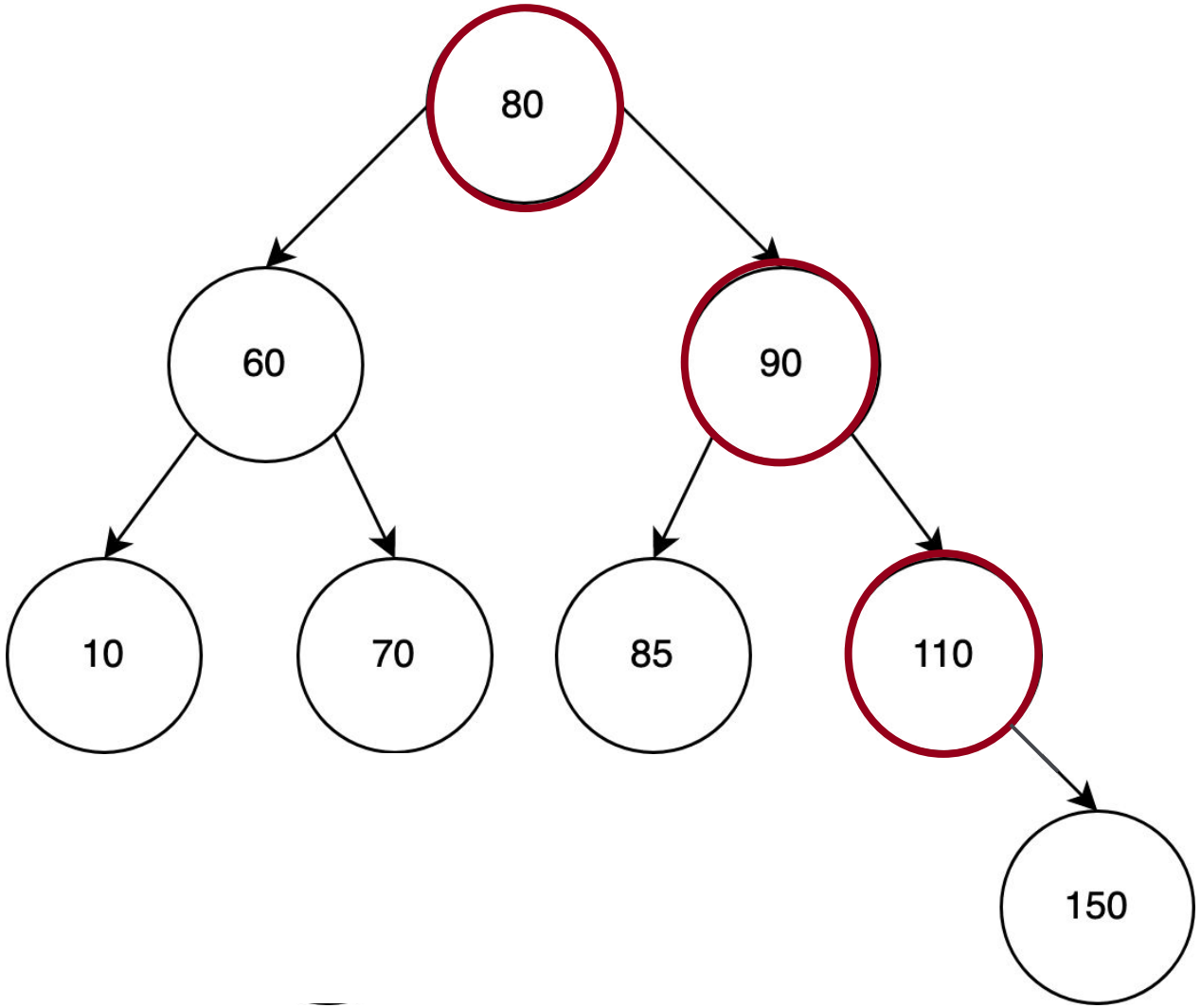
# Today's Lecture

1. Binary Search Trees
2. Search
3. **Insertion**
4. Removal
5. Summary

# Binary Search Trees: Insertion

Insertion must maintain the properties of a BST!

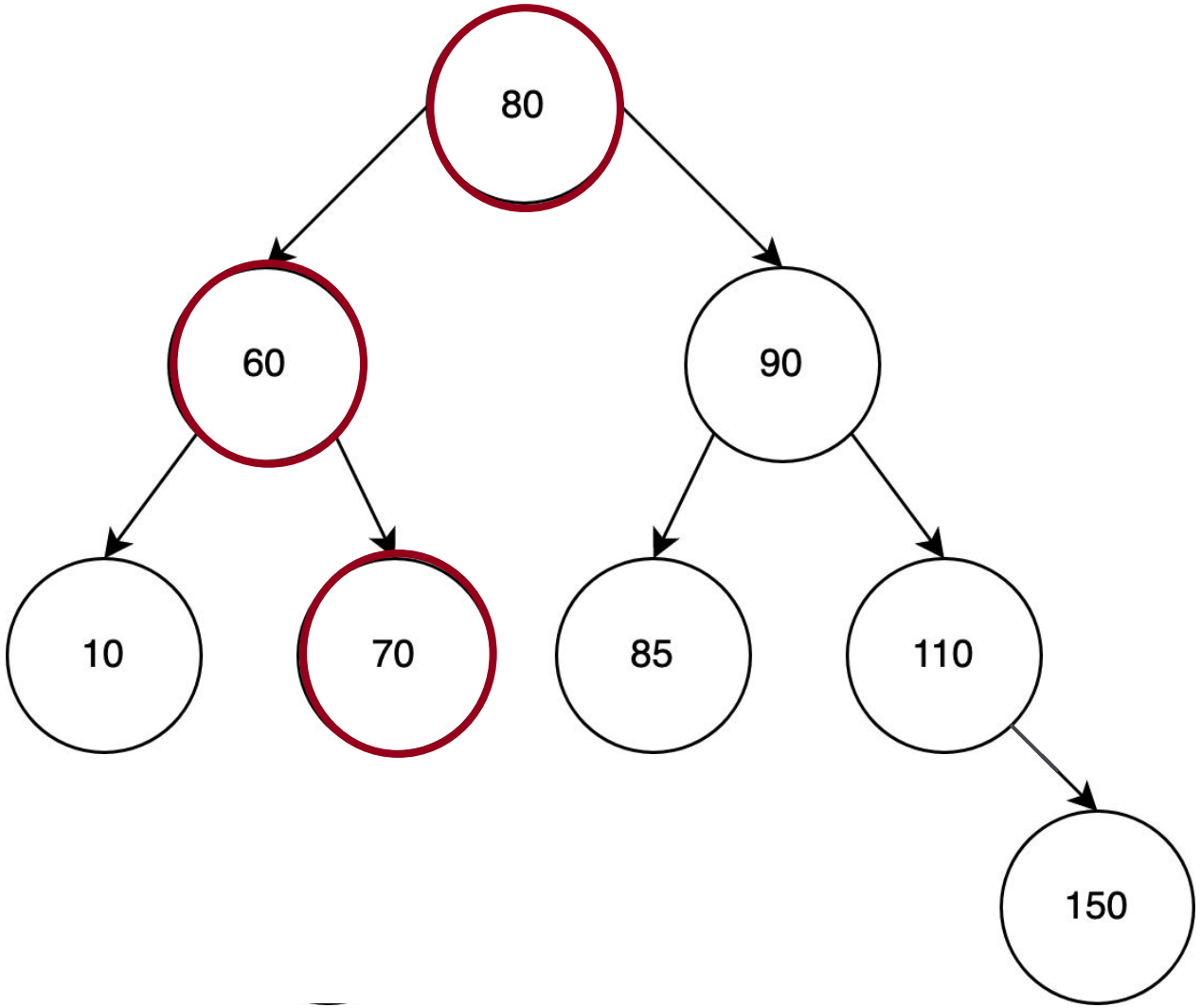
**Insert**: 150



# Binary Search Trees: Insertion

Insertion must maintain the properties of a BST!

**Insert**: 64




Complexity?
O(log n)

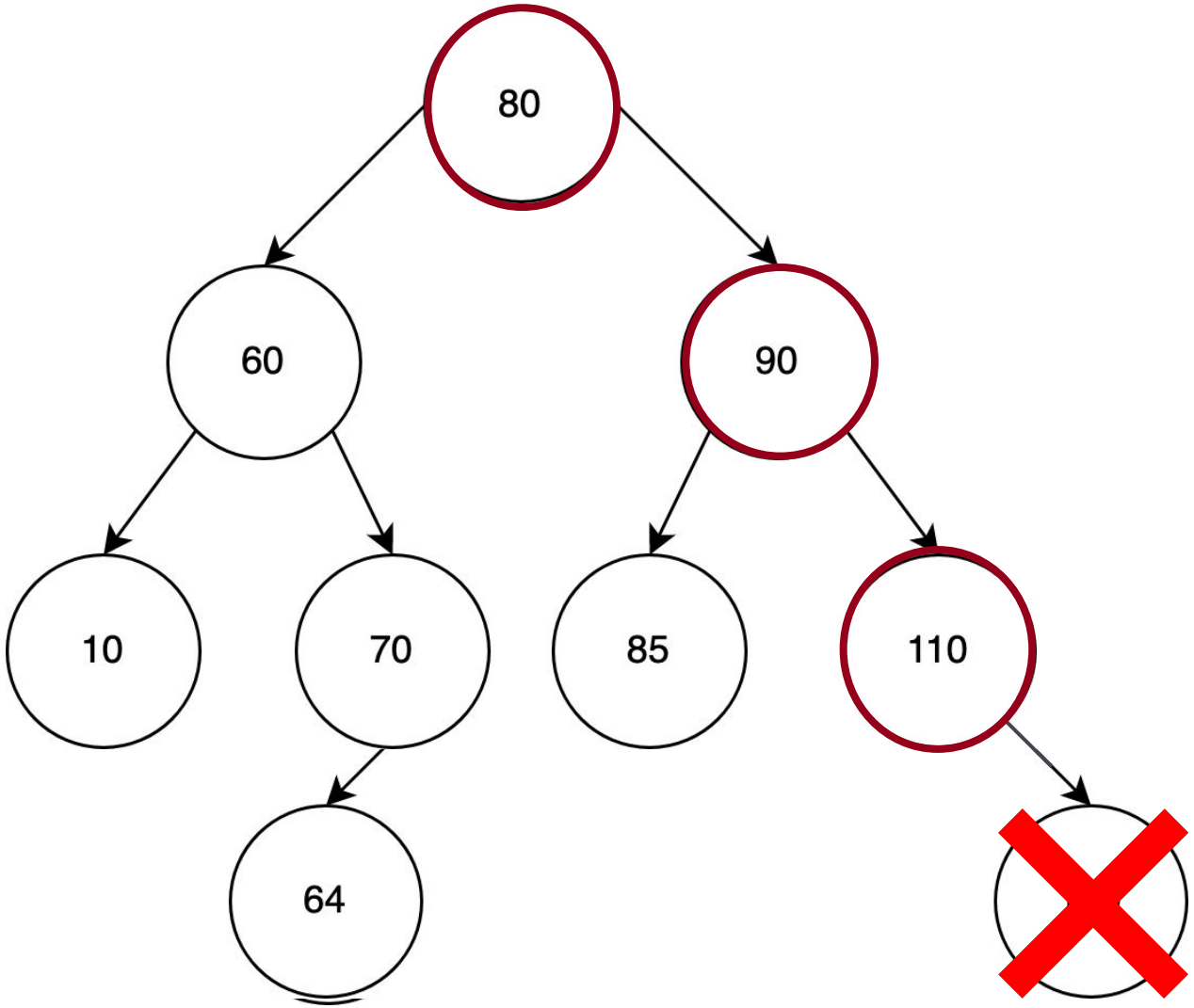
# Today's Lecture

1. Binary Search Trees
2. Search
3. Insertion
4. **Removal**
5. Summary

# Binary Search Trees: Deletion

Deletion must maintain the properties of a BST!

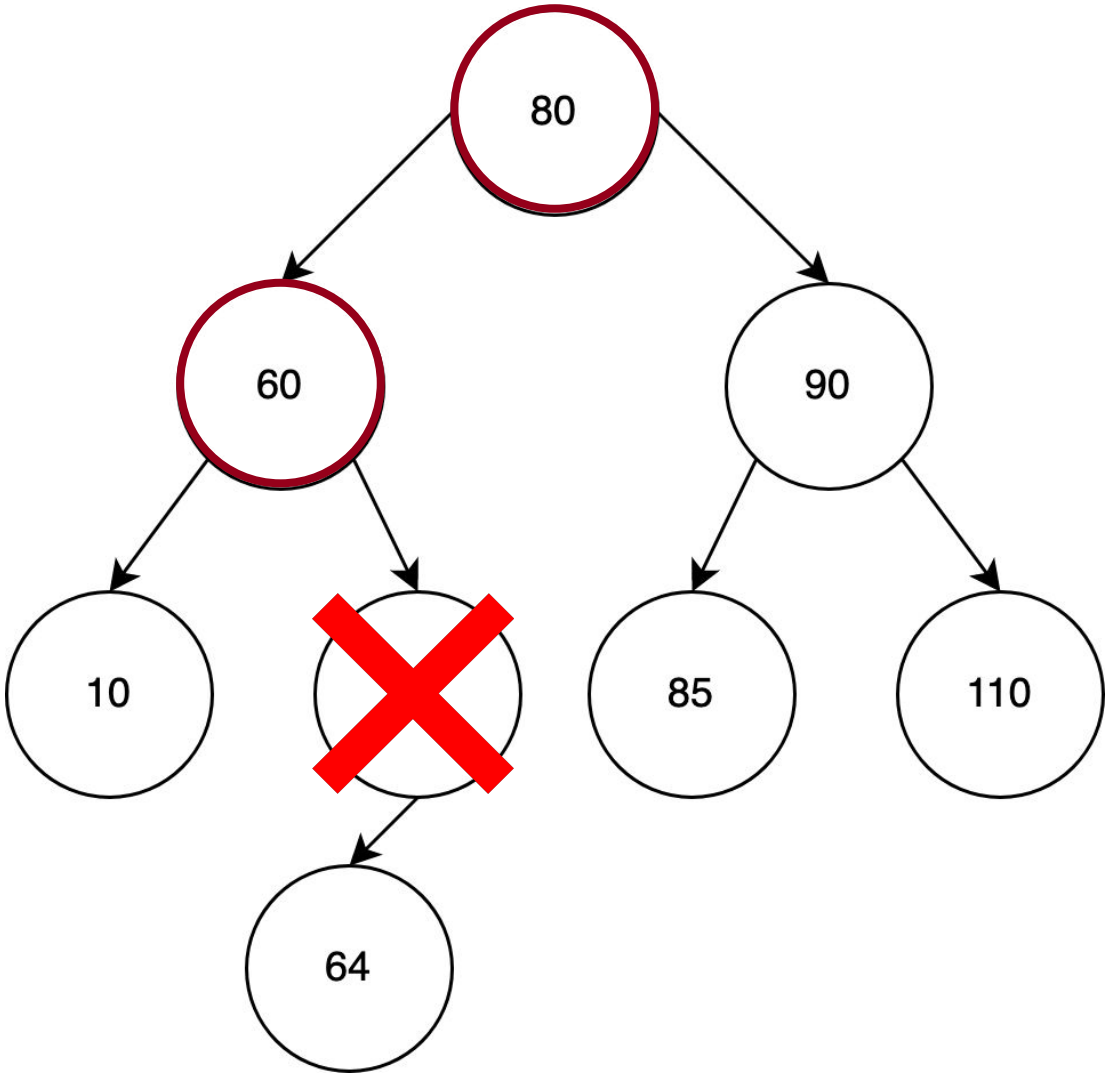
**Delete**: 150



# Binary Search Trees: Deletion

Deletion must maintain the properties of a BST!

**Delete**: 70



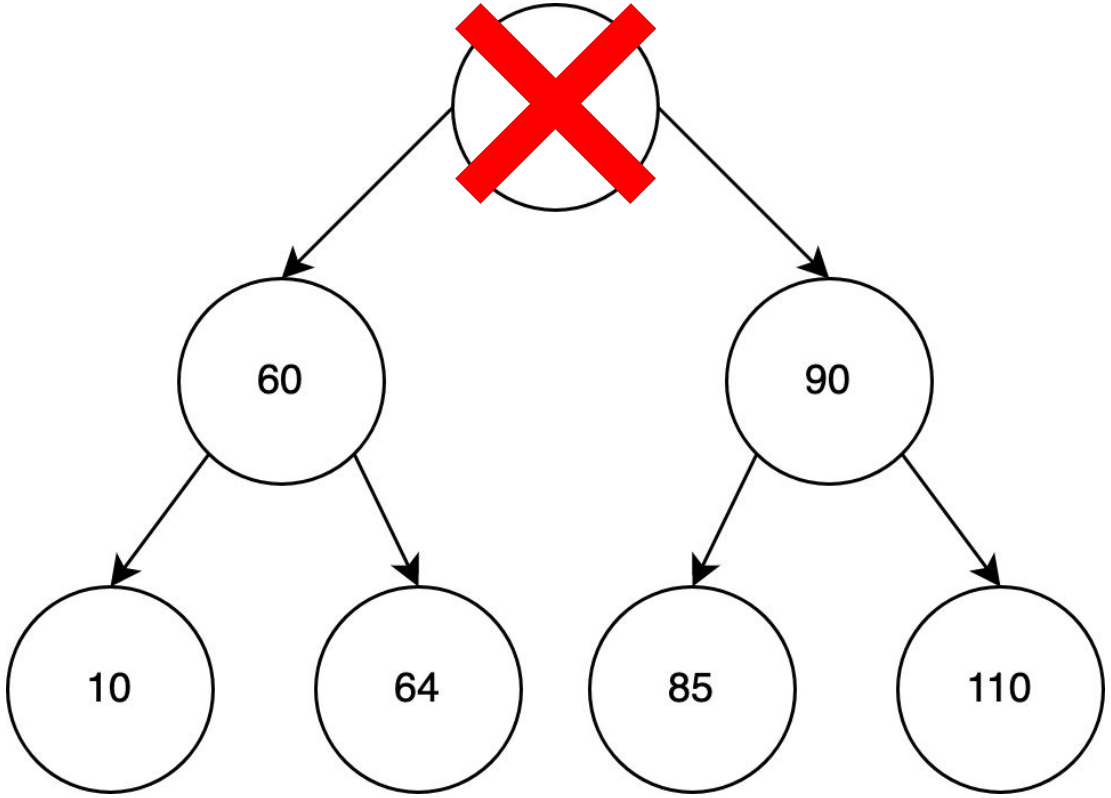
# Binary Search Trees: Deletion

## Deletion must maintain the properties of a BST!

**Delete**: 80

At **each node** with value **k**

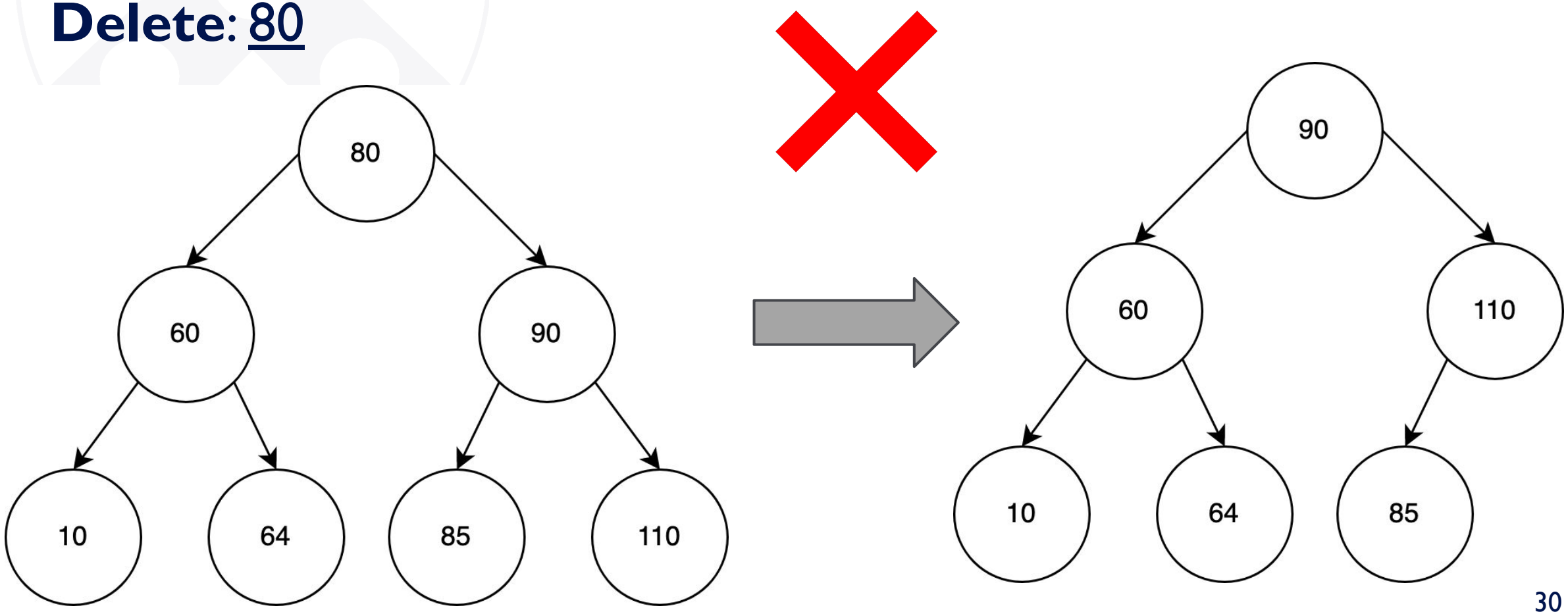
- Left subtree contains only nodes with value **lesser** than **k**
- Right subtree contains only nodes with value **greater** than **k**
- Both subtrees are a **binary search tree**



# Binary Search Trees: Deletion

## Replace with 90?

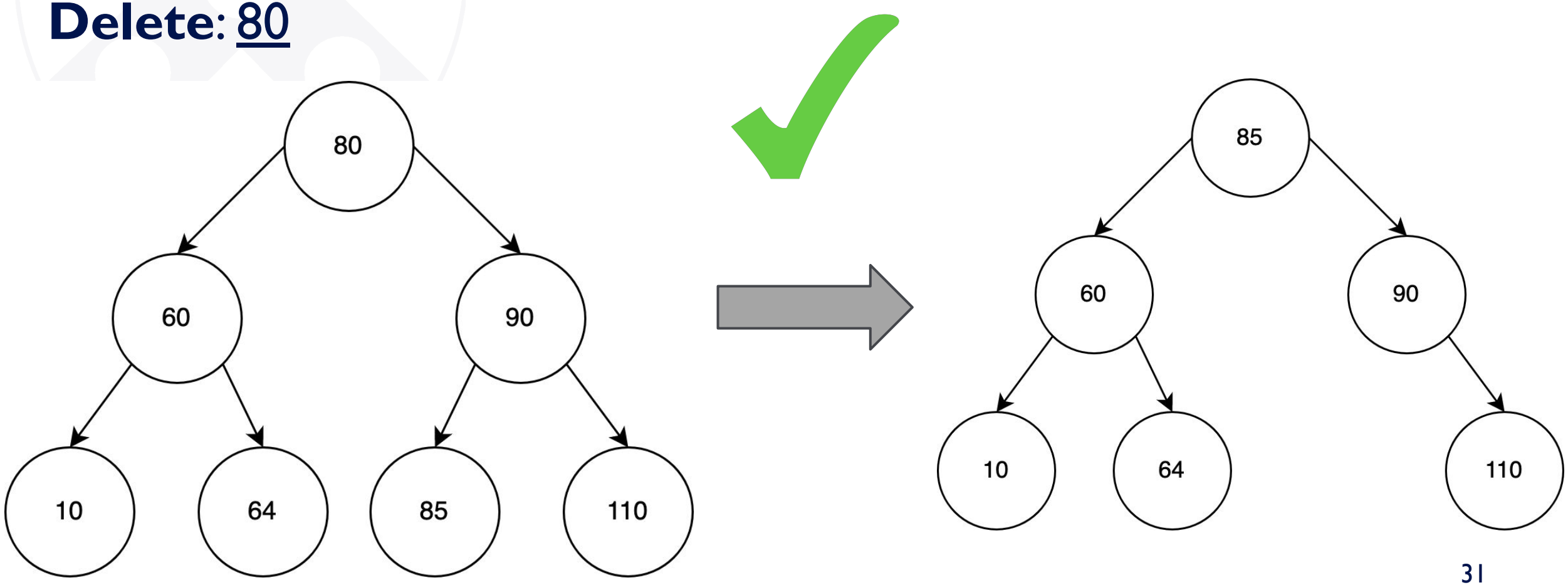
**Delete:** 80



# Binary Search Trees: Deletion

## Replace with 85?

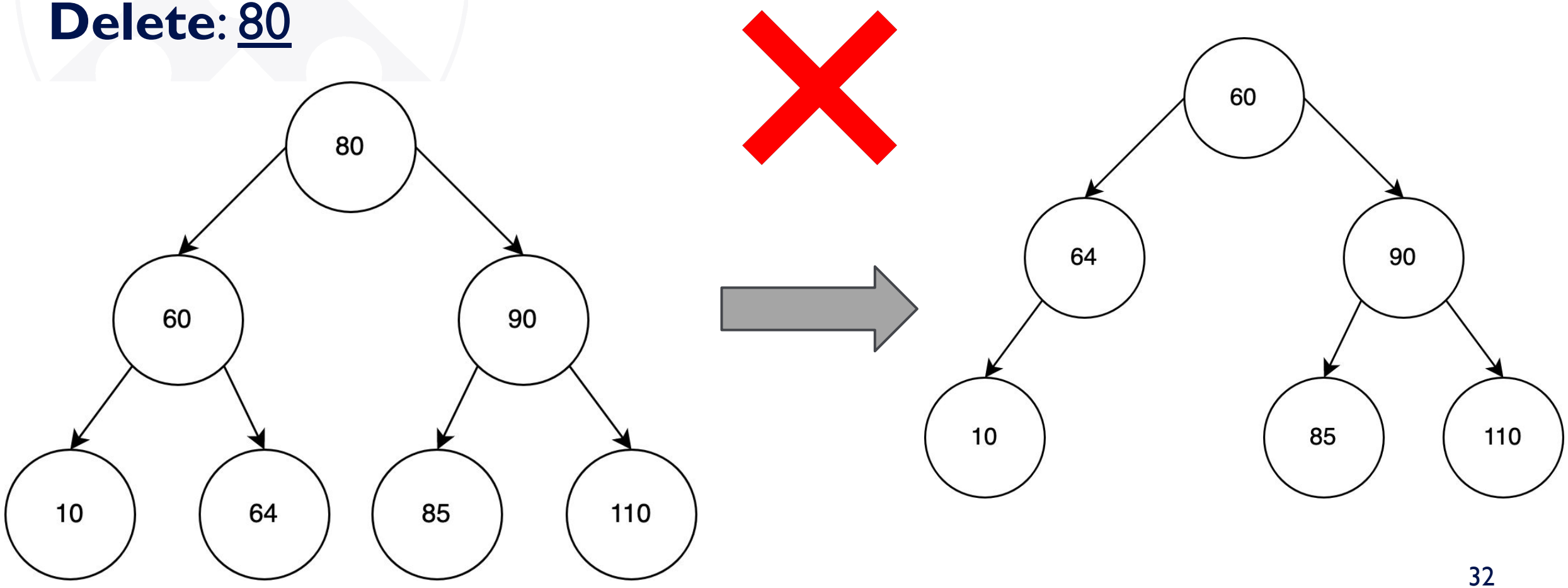
**Delete:** 80



# Binary Search Trees: Deletion

## Replace with 60?

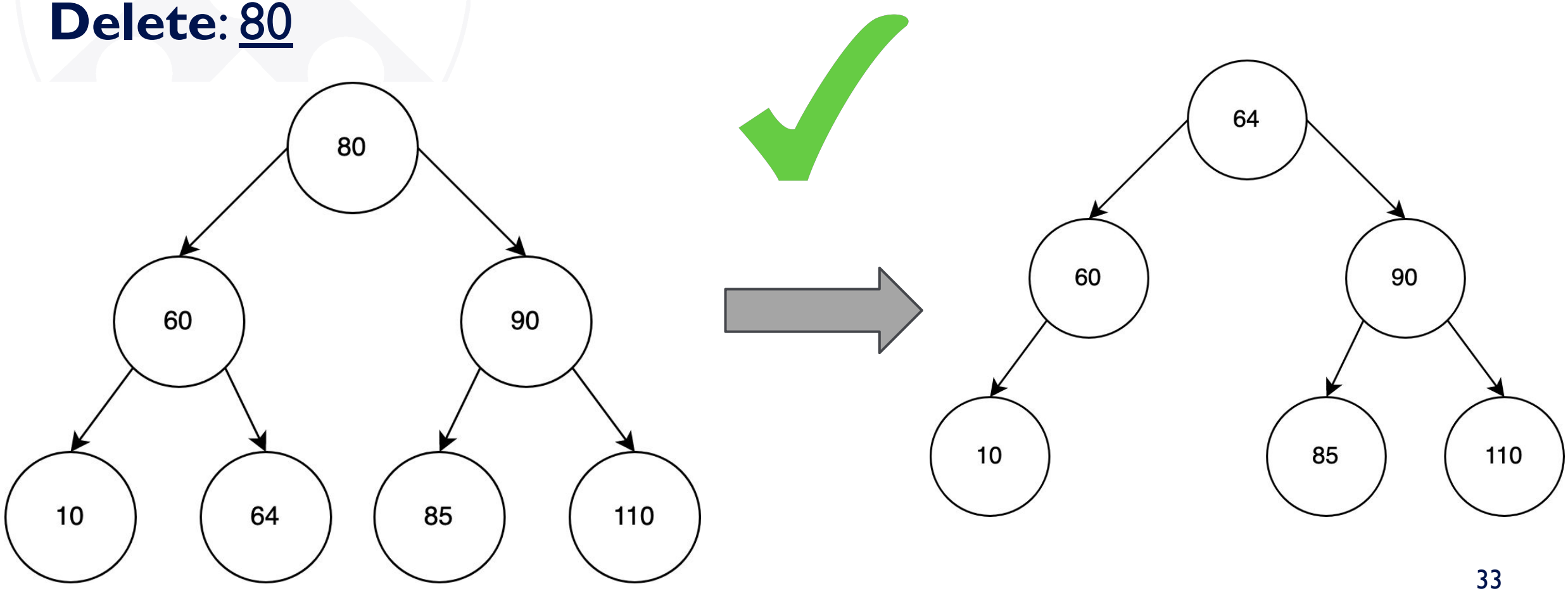
**Delete:** 80



# Binary Search Trees: Deletion

## Replace with 64?

**Delete:** 80



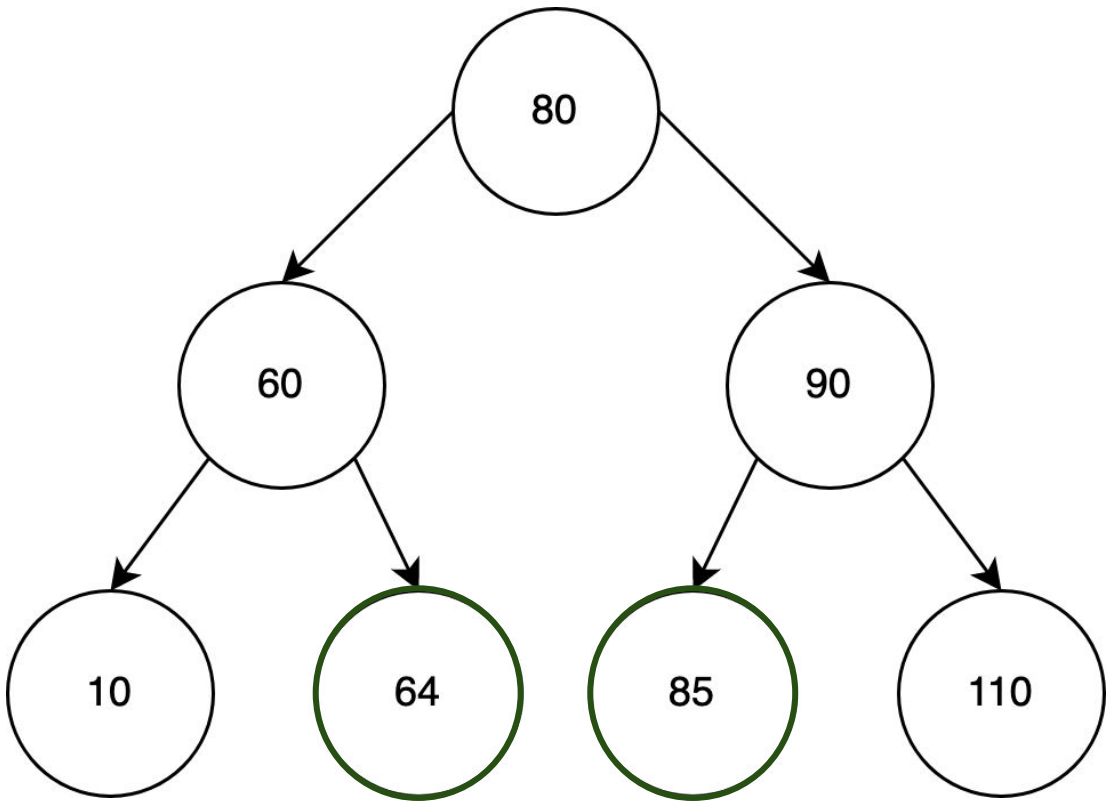
# Binary Search Trees: Deletion

## Deletion must maintain the properties of a BST!

**Delete**: 80

Replace deleted node with either:

1. Smallest value in right subtree
2. Largest value in left subtree



# Binary Search Trees: Deletion

Complexity?

Case 1: Removing a **leaf node**

O(log n)

Case 2: Removing a **node with one child**

O(log n)

Case 3: Removing a **node with two children**

O(log n)

# What can go wrong?

Complexity?

**Search**

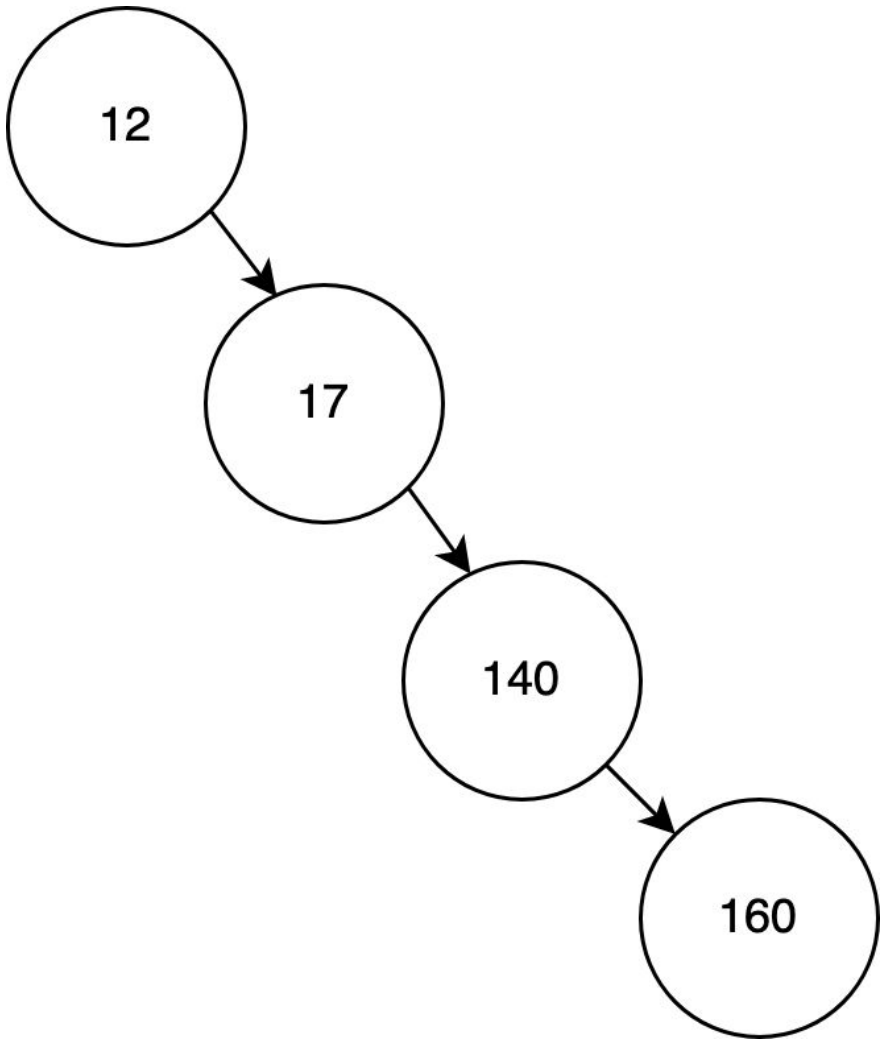
O(n)

**Insertion:**

O(n)

**Deletion:**

O(n)



# Tree Traversals

# Binary Tree Traversals

Traversal visits all nodes in a tree in some order

Inorder:

left subtree, current, right subtree

will  print “in order” (increasing values)

Preorder:

current, left subtree, right subtree

Postorder:

left subtree, right subtree, current

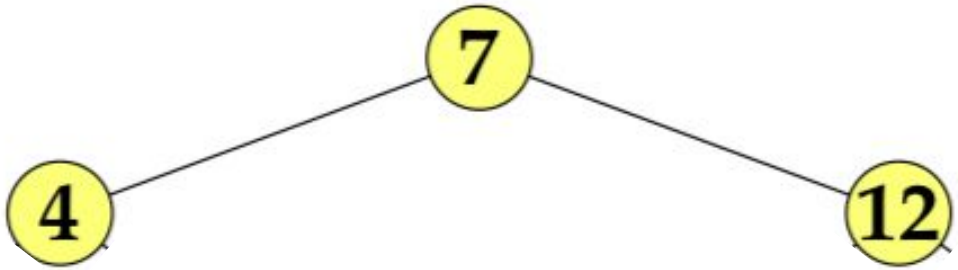
# In Order Traversal

1. Move left until you reach a node without a left child
2. Print the current node
3. Move right

# Inorder Example 1

What would the in-order traversal be here?

left subtree, current, right subtree



# Inorder Example 2

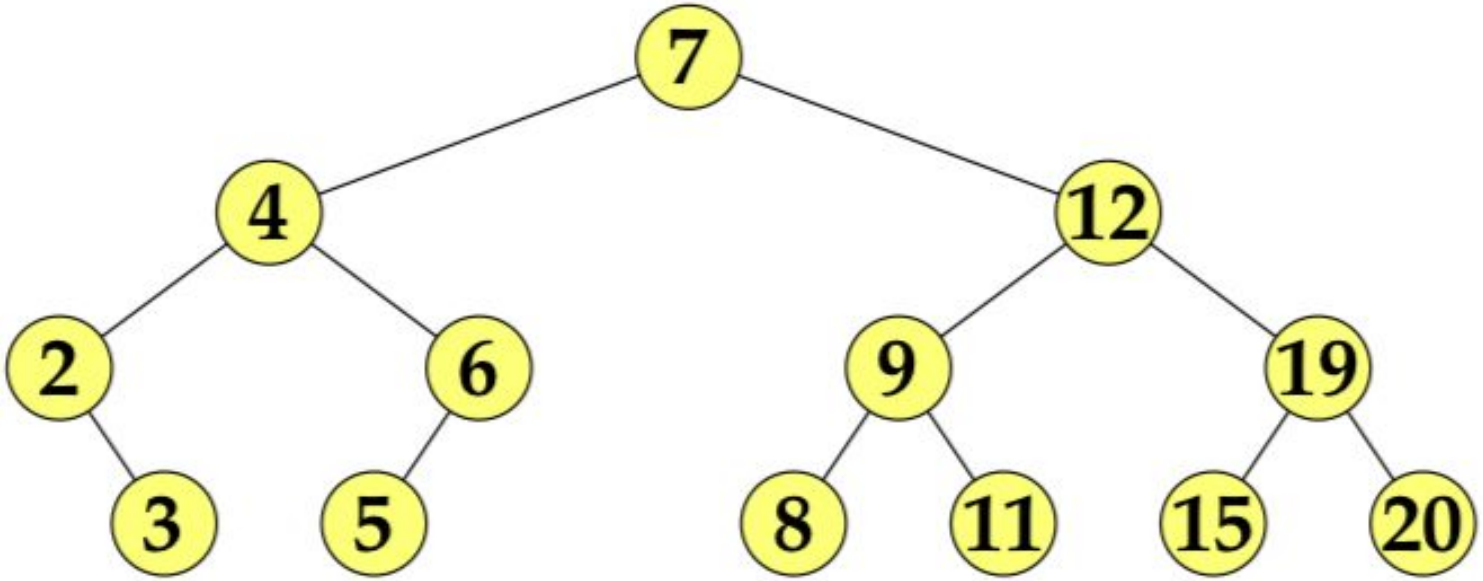
- Larger tree (Height > 1)

- **Process entire left subtree first**
  - **bottom most left node**
  - **current**
  - **bottom most right node**

# Inorder Example 2

What would the in-order traversal be here?

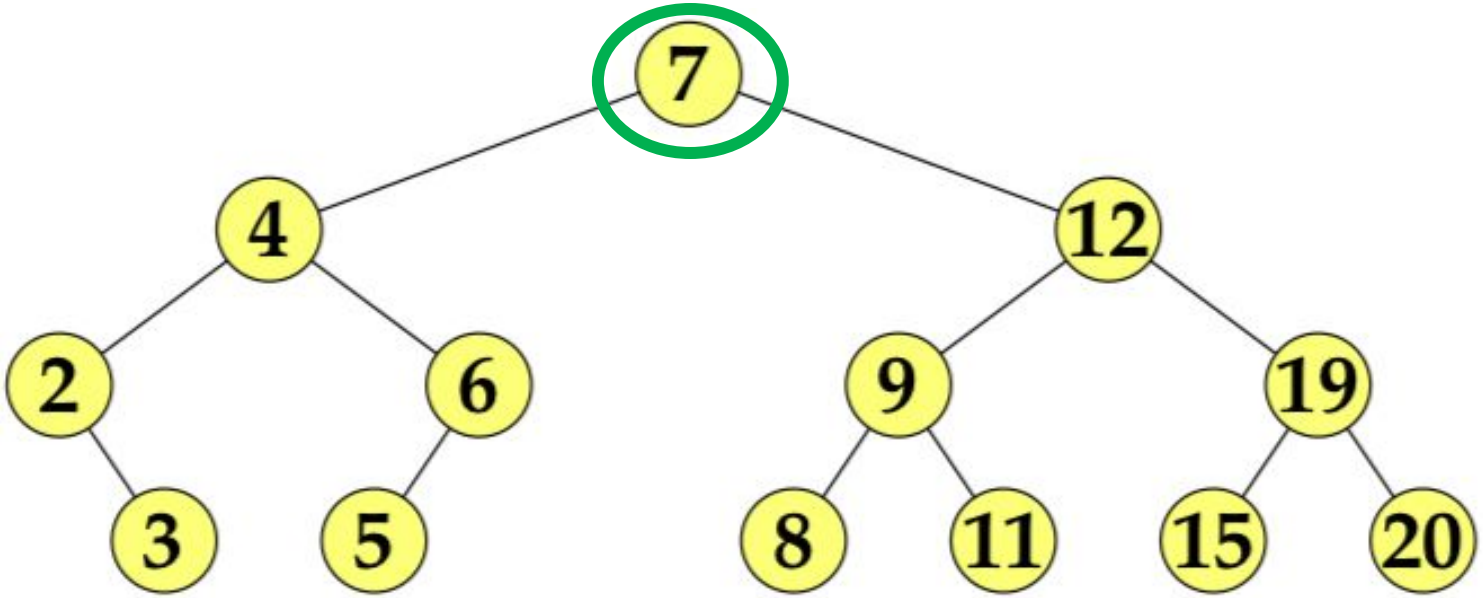
left subtree, current, right subtree



# Inorder

What would the in-order traversal be here?

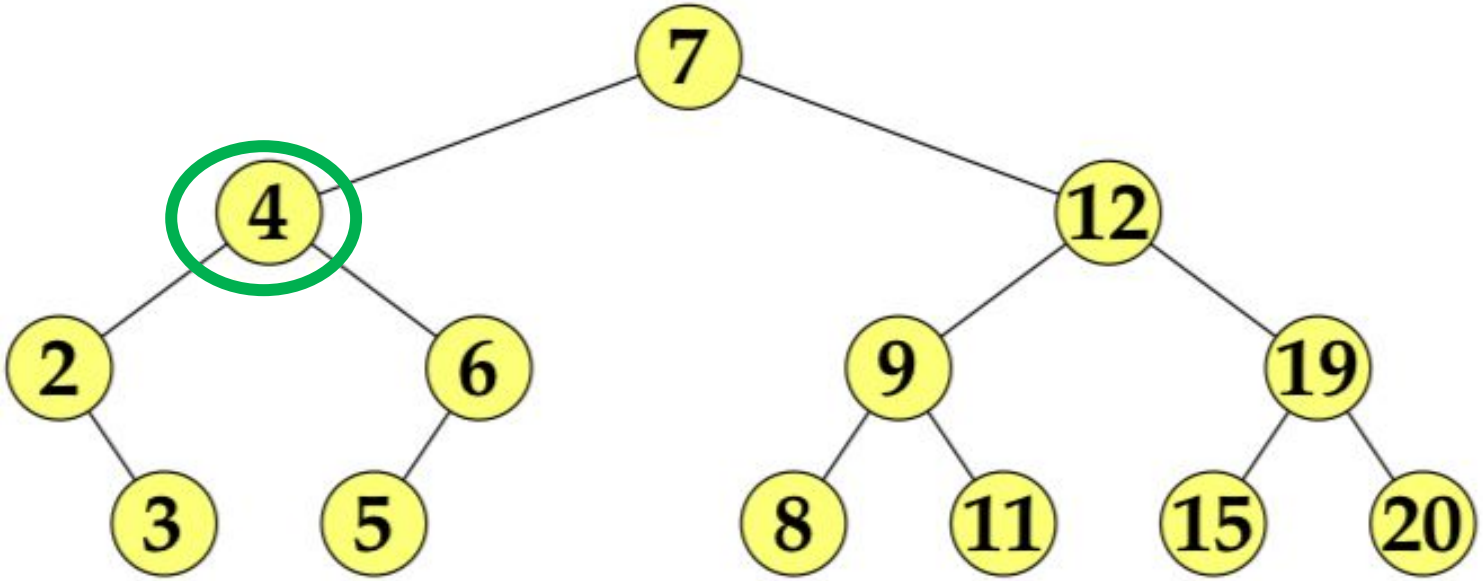
left subtree, current, right subtree



# Inorder

What would the in-order traversal be here?

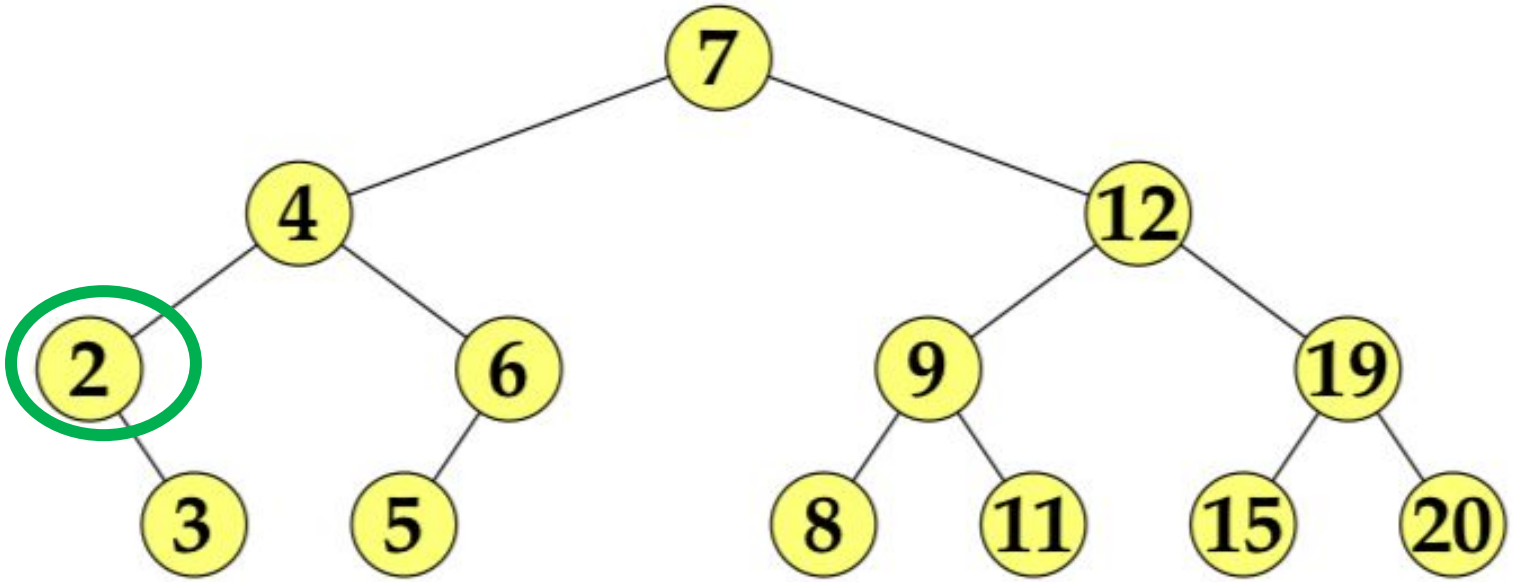
left subtree, current, right subtree



# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

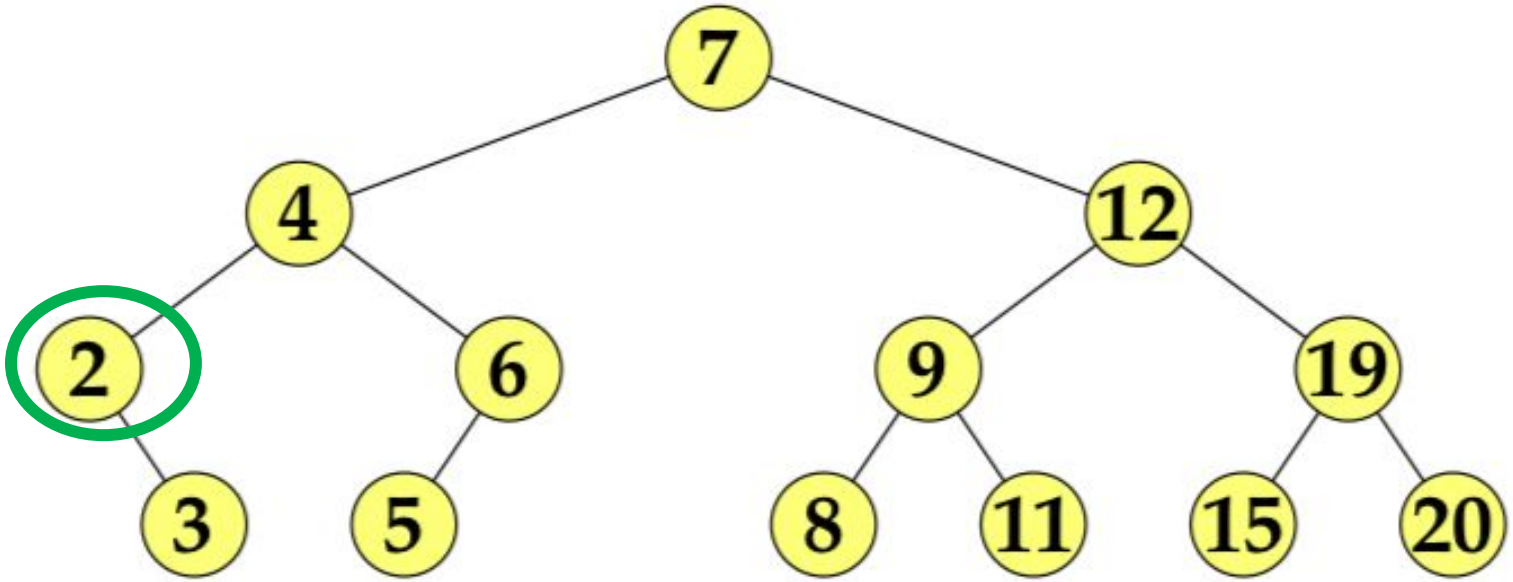


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2,

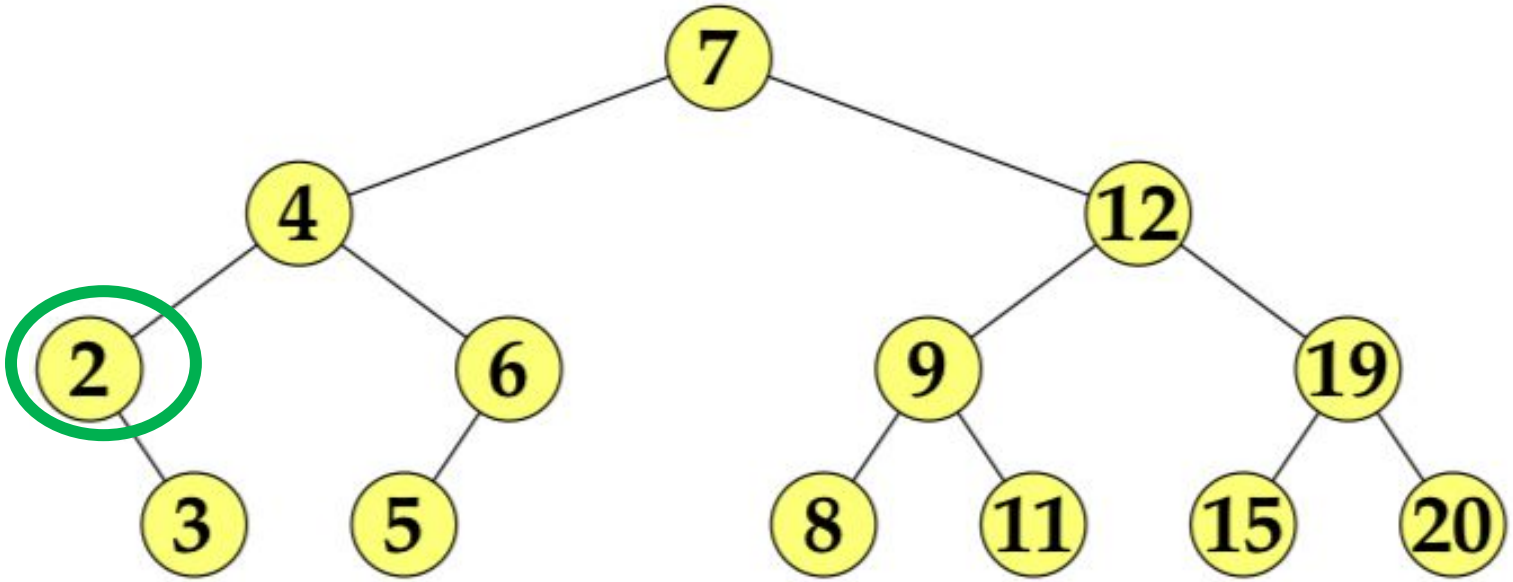


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2,

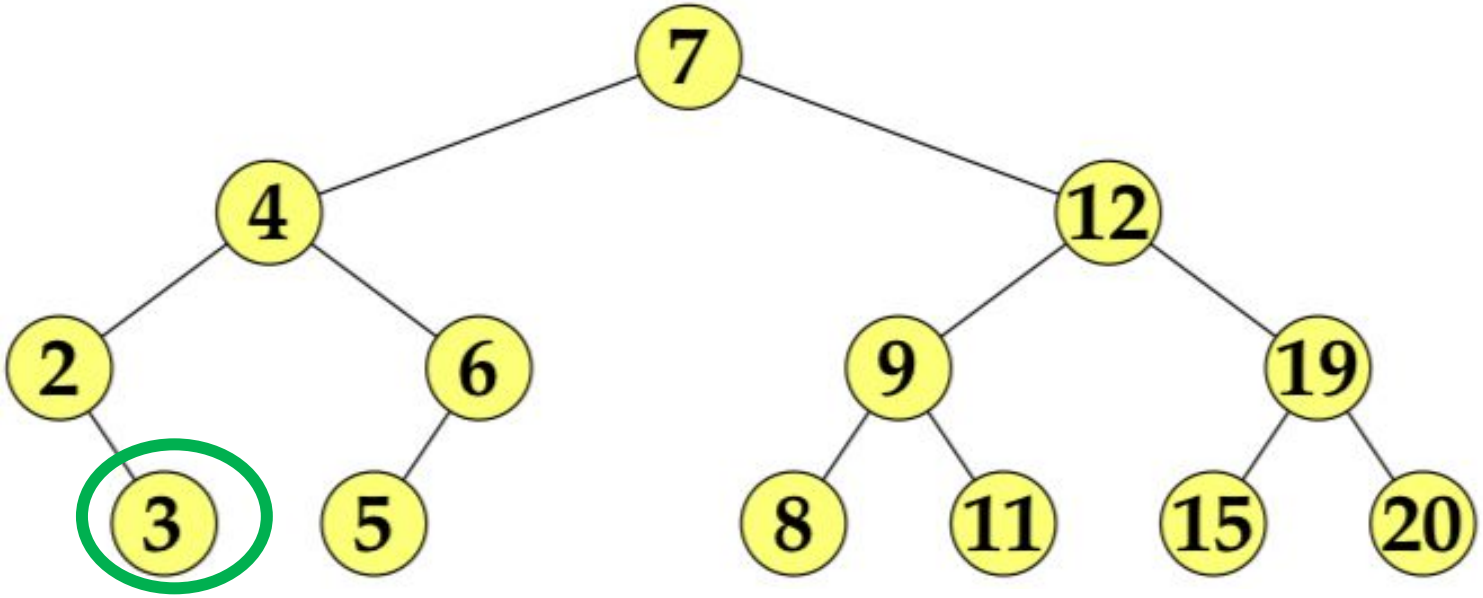


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2,

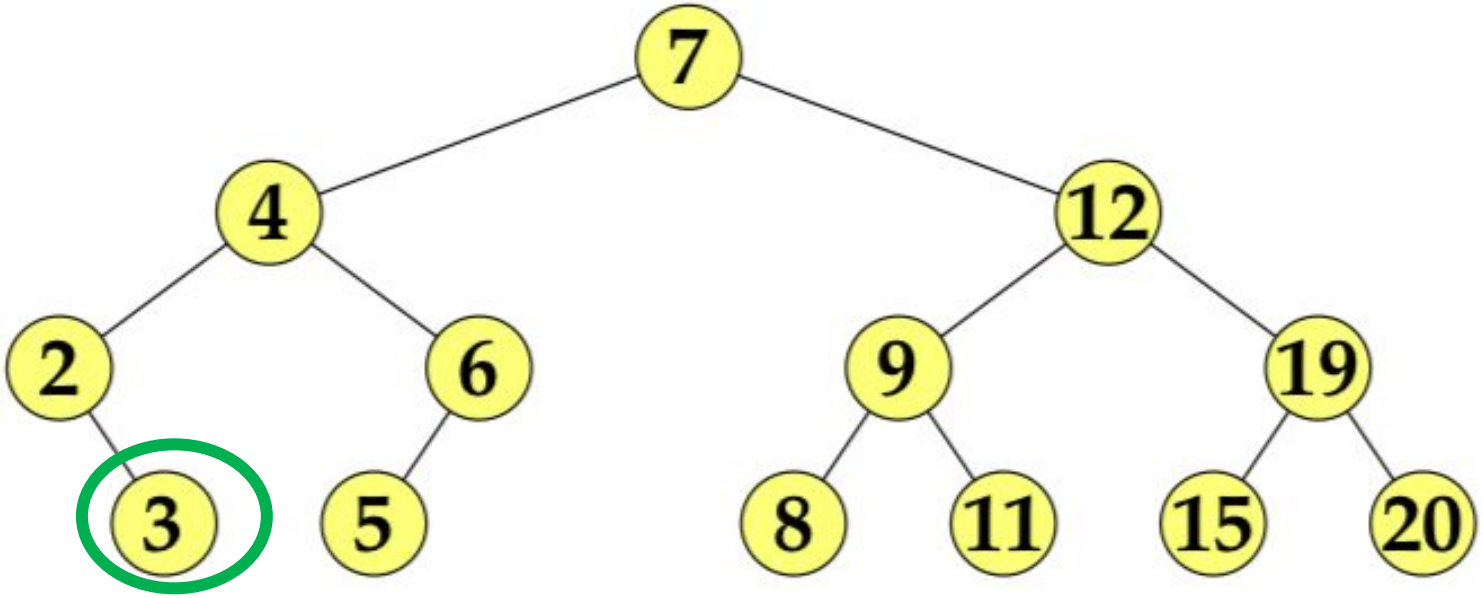


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3

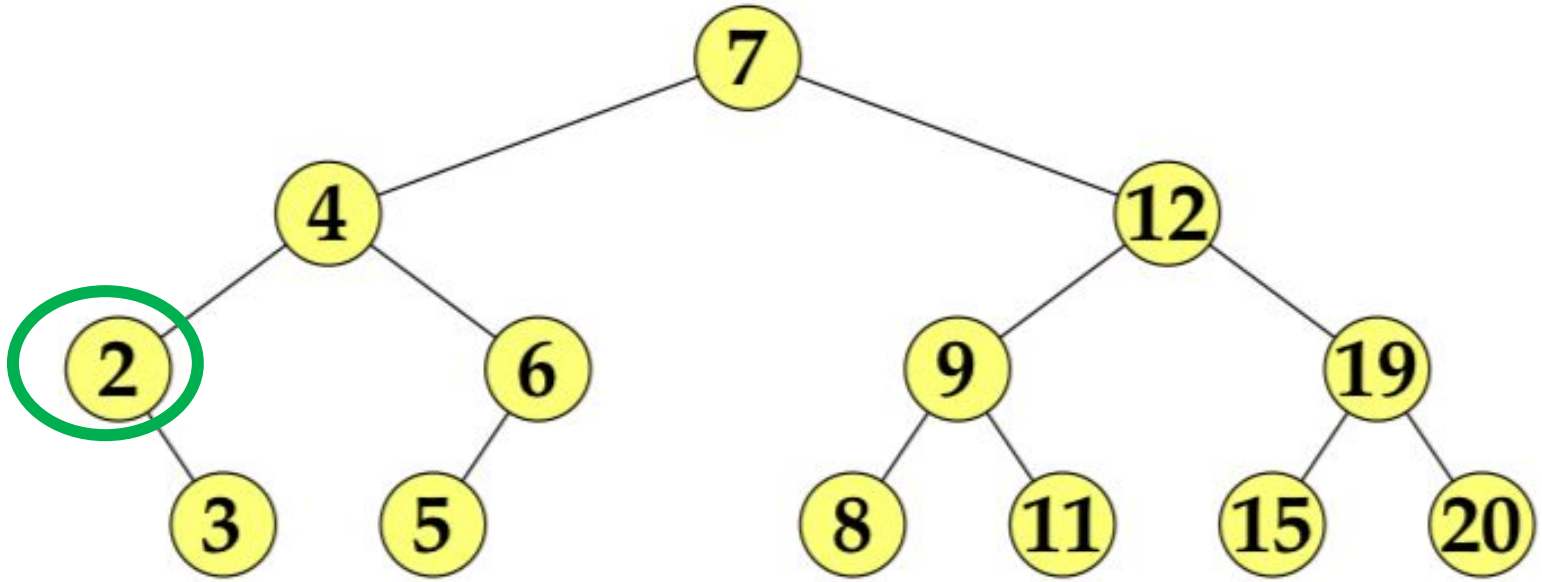


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3

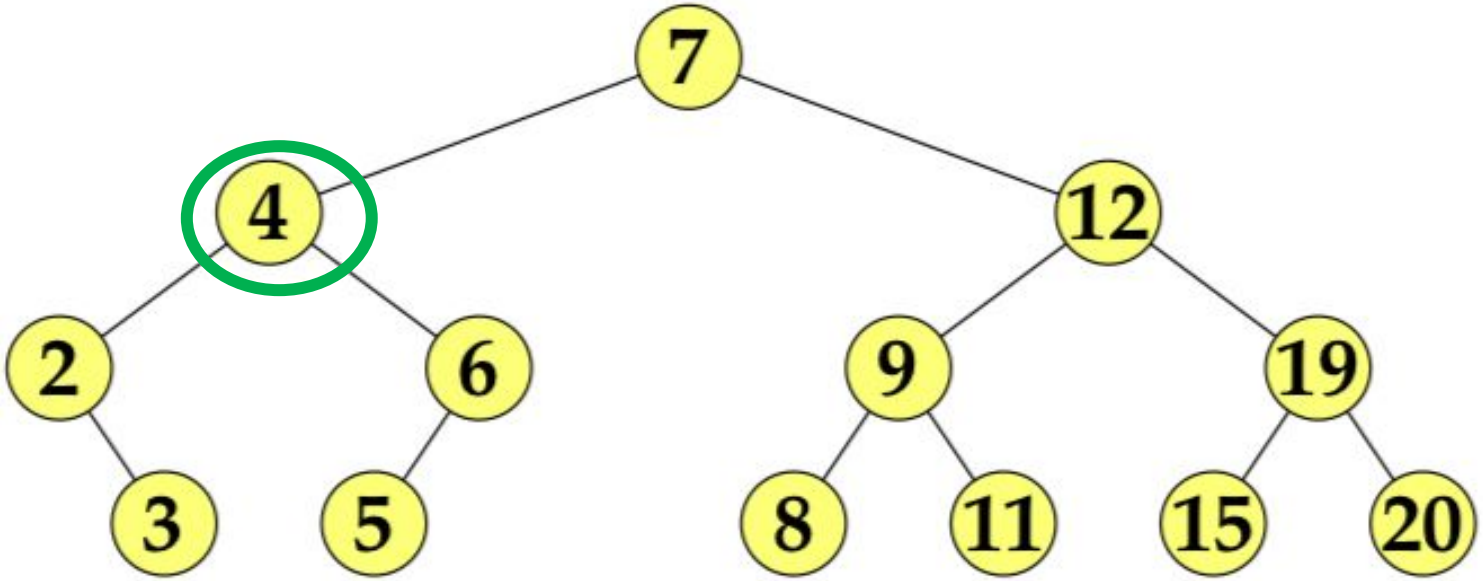


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3

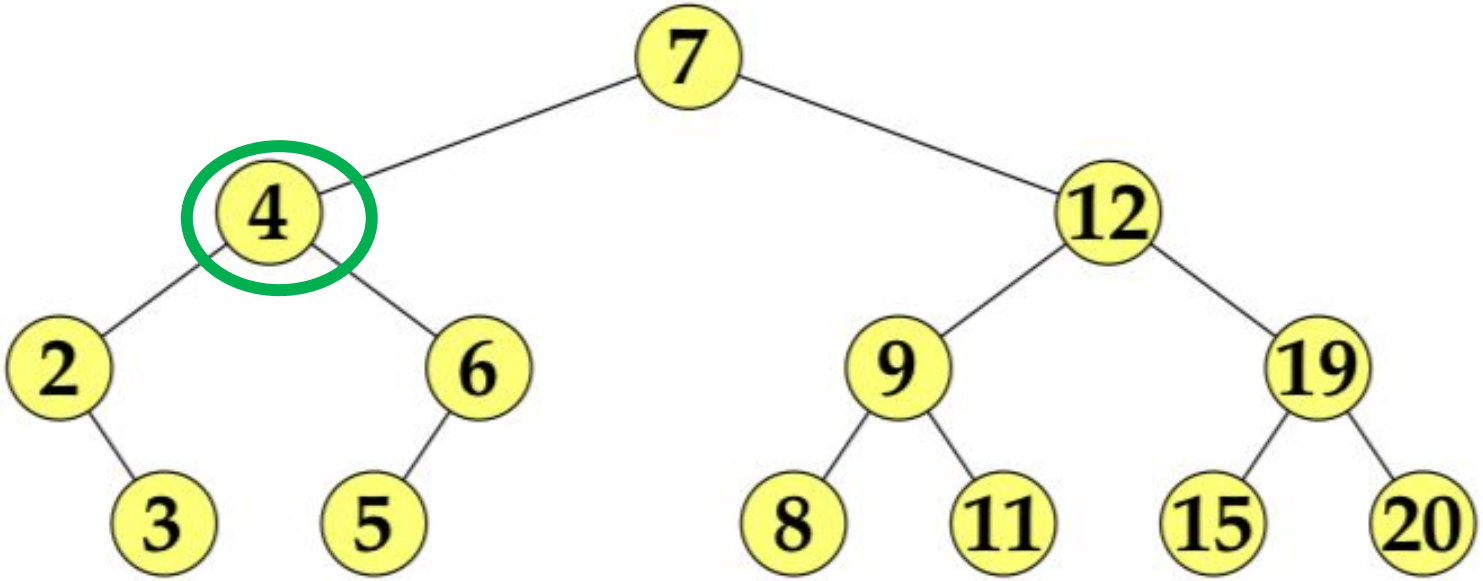


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4

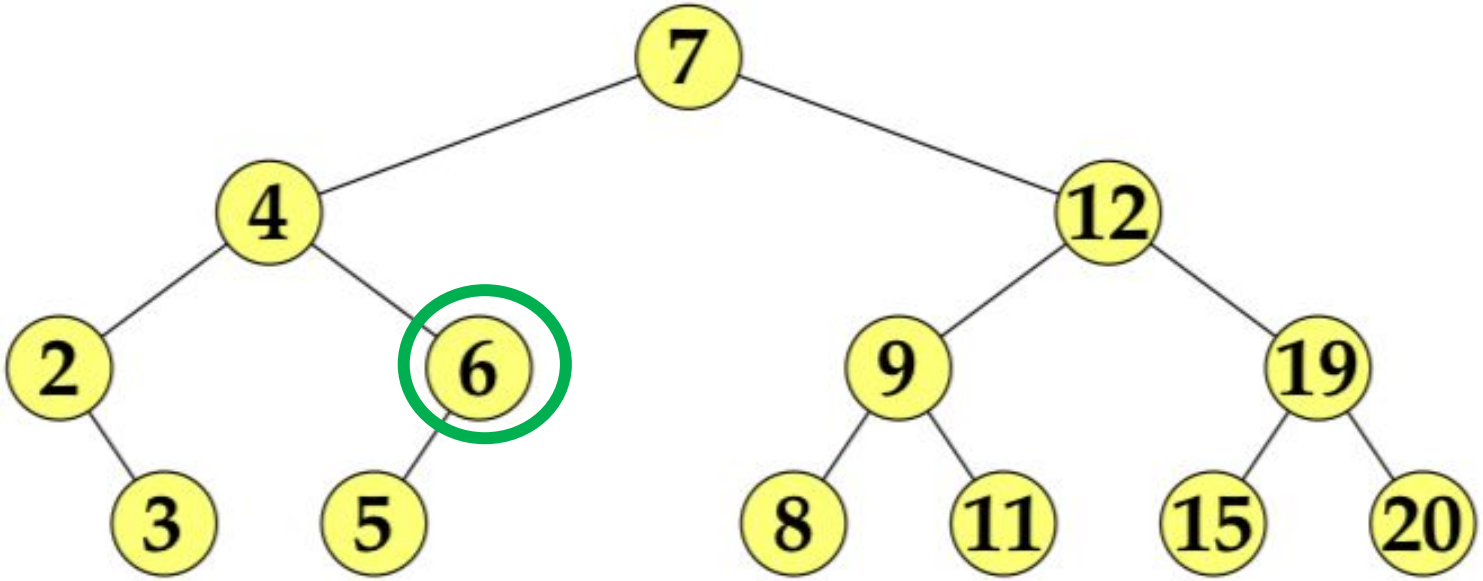


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4

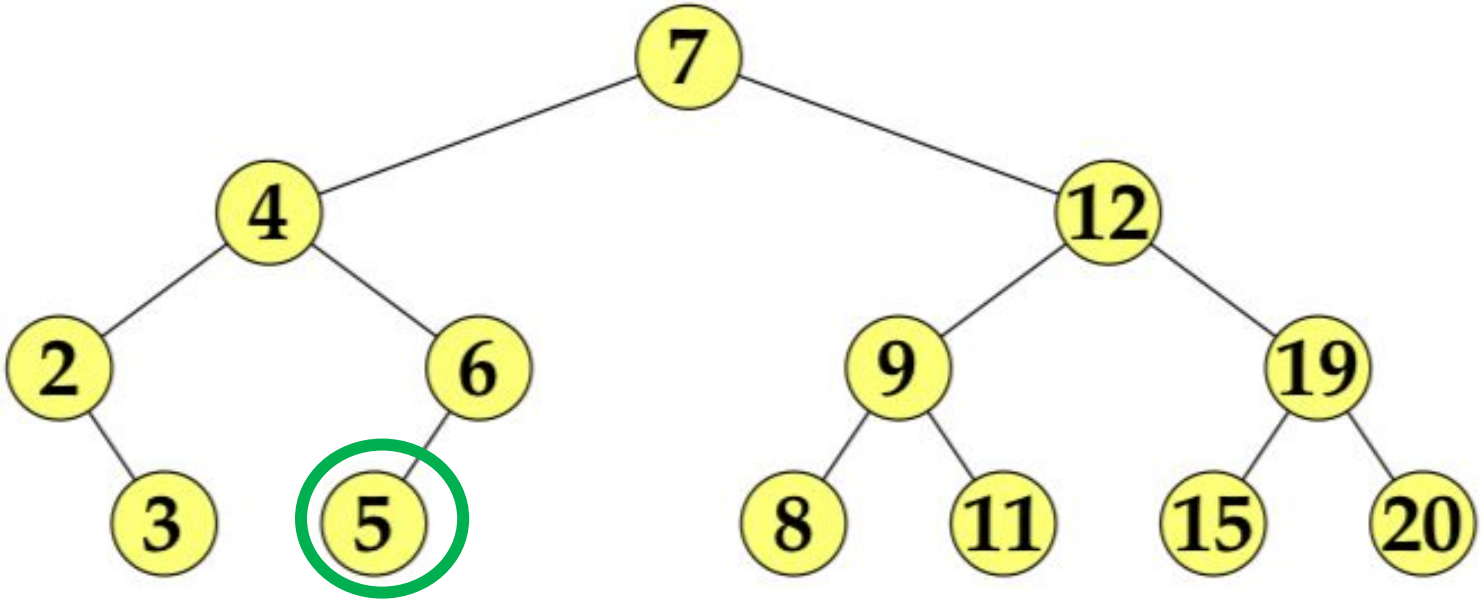


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4

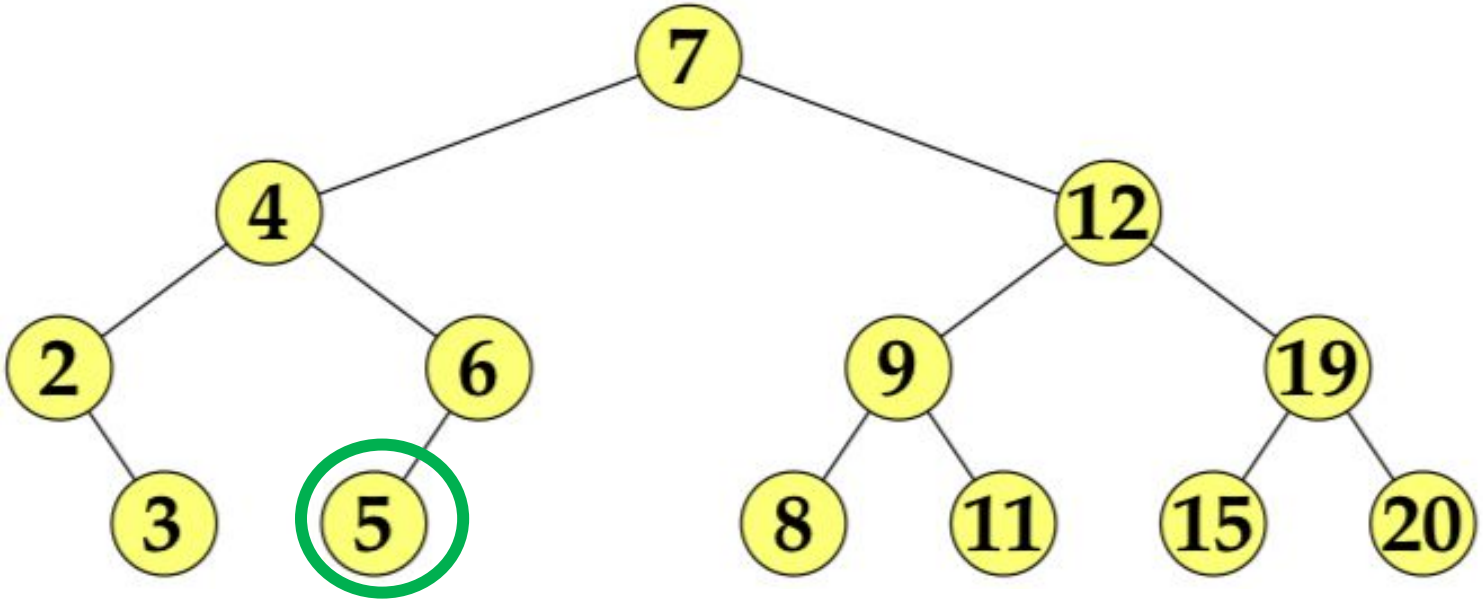


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5

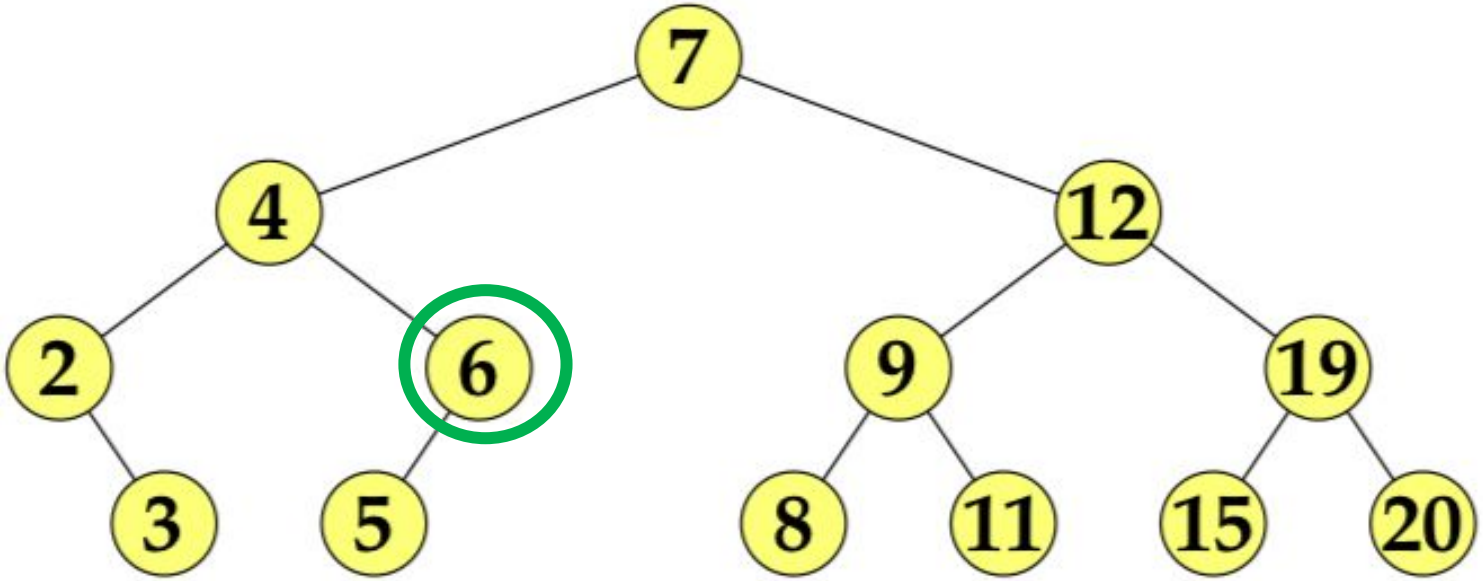


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5

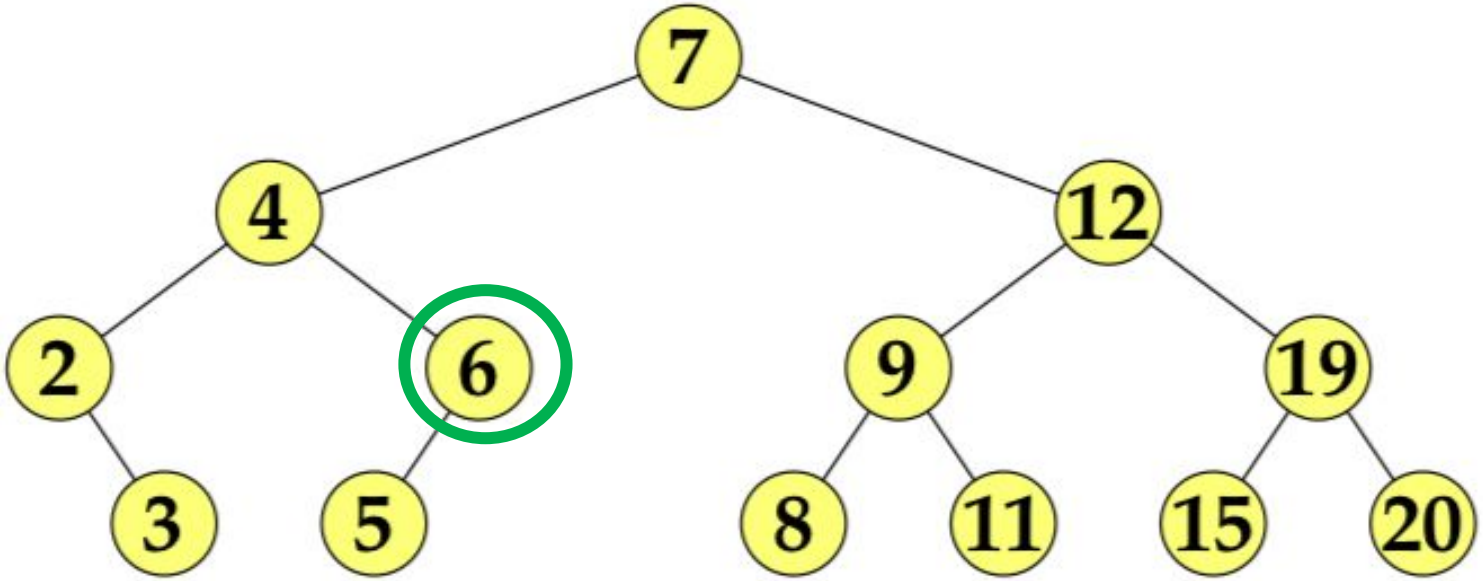


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6

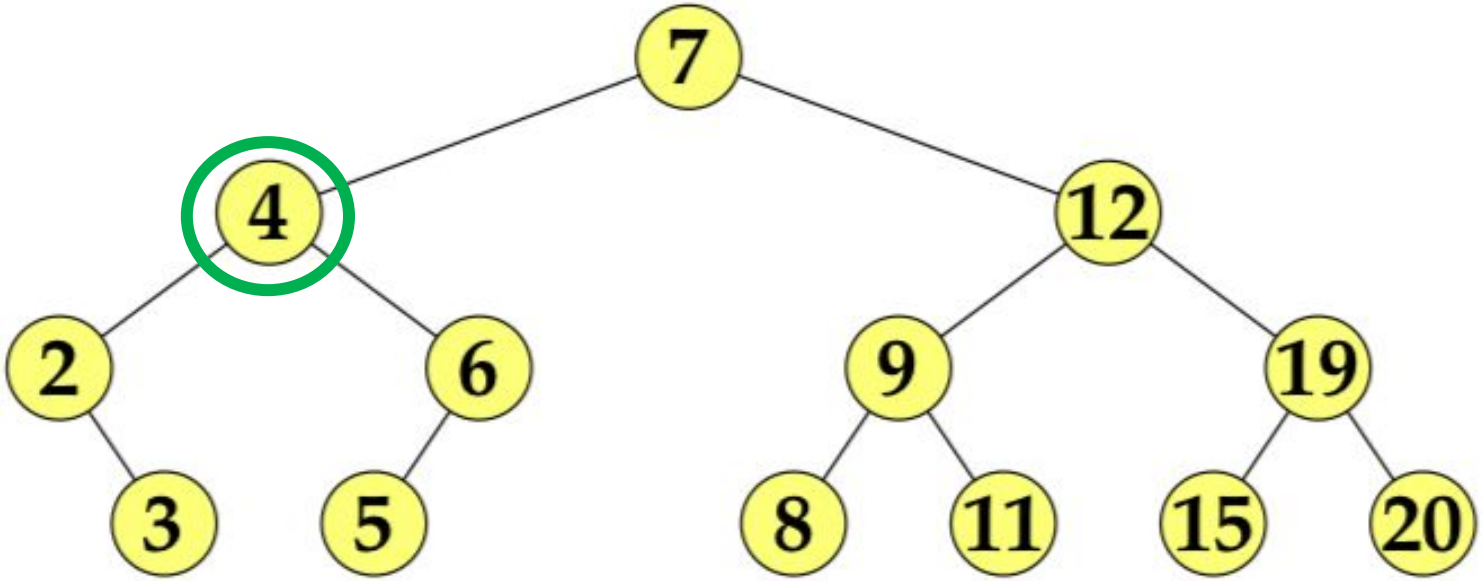


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6

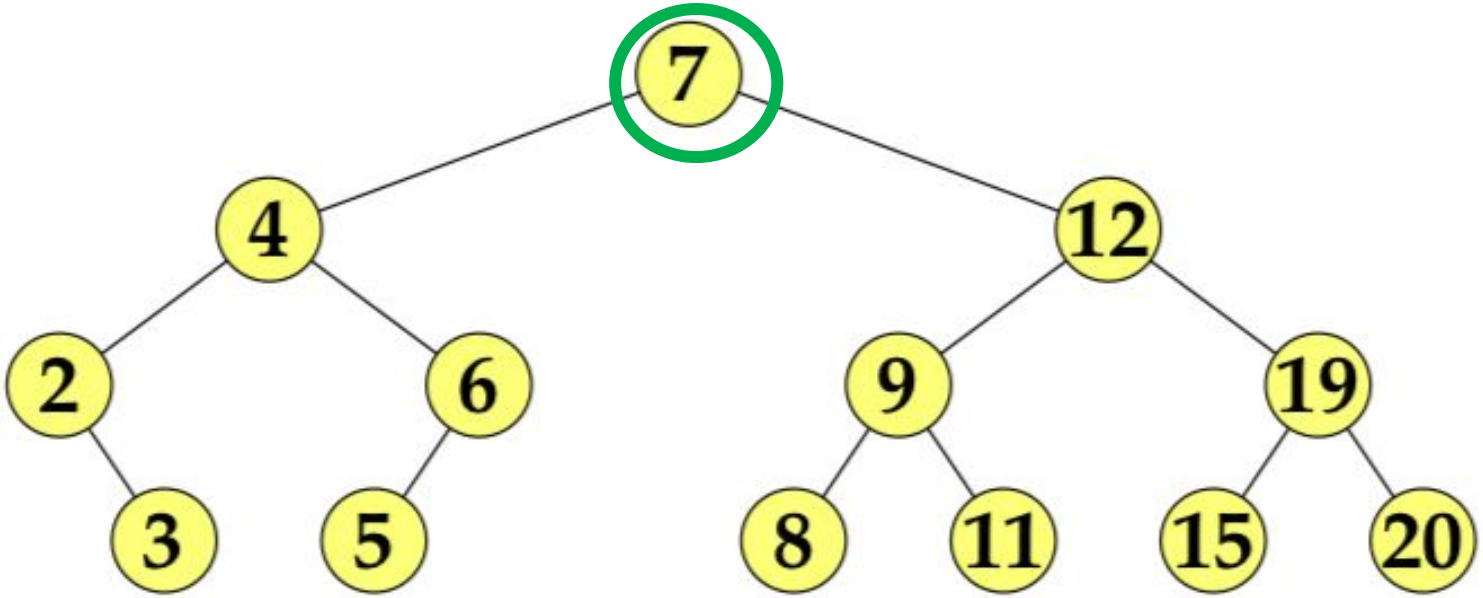


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6

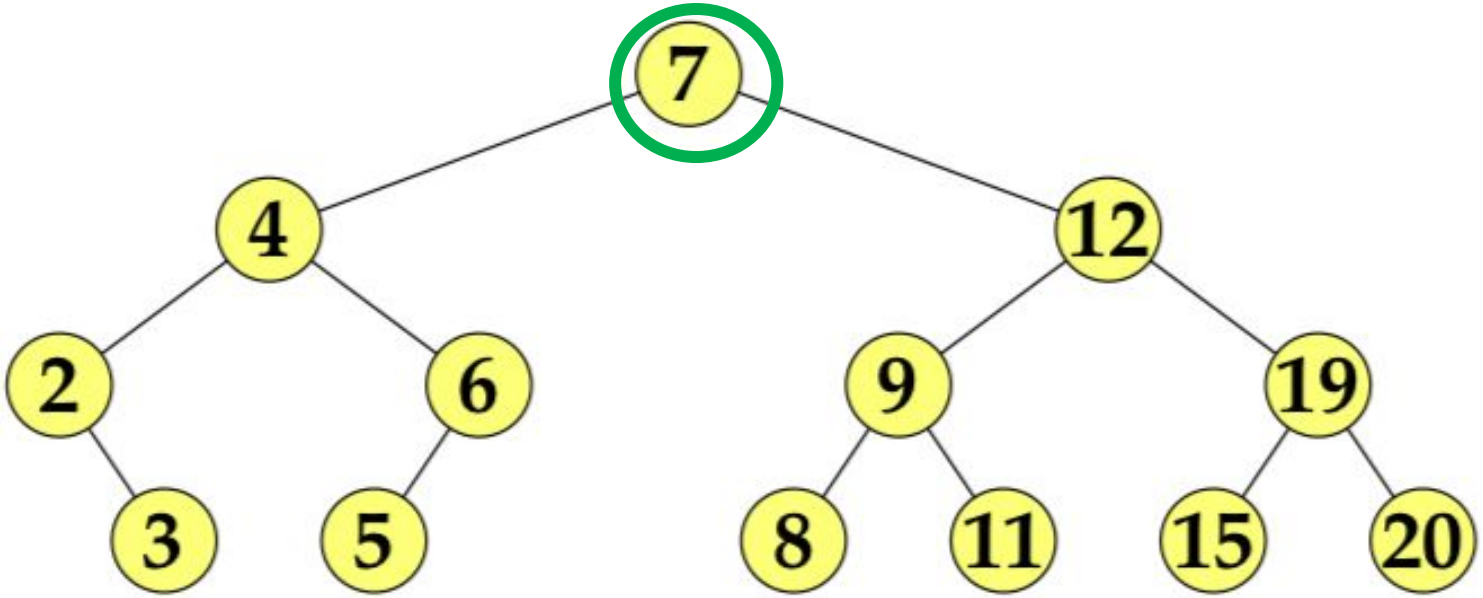


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7

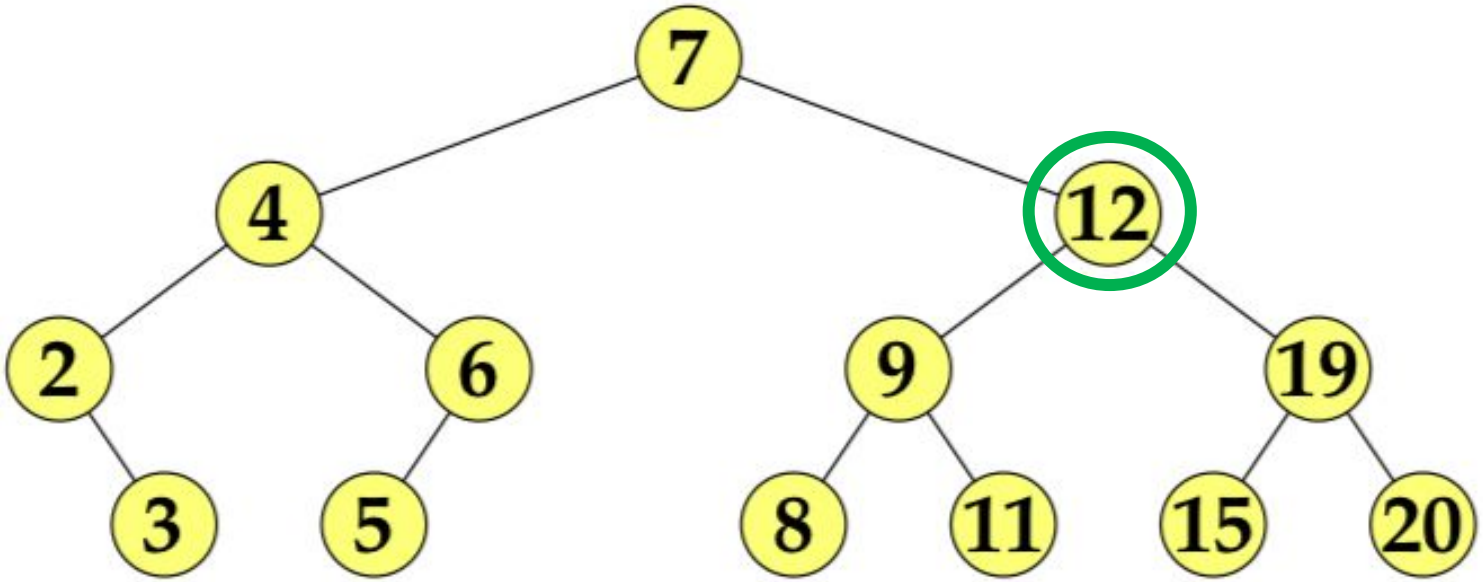


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7

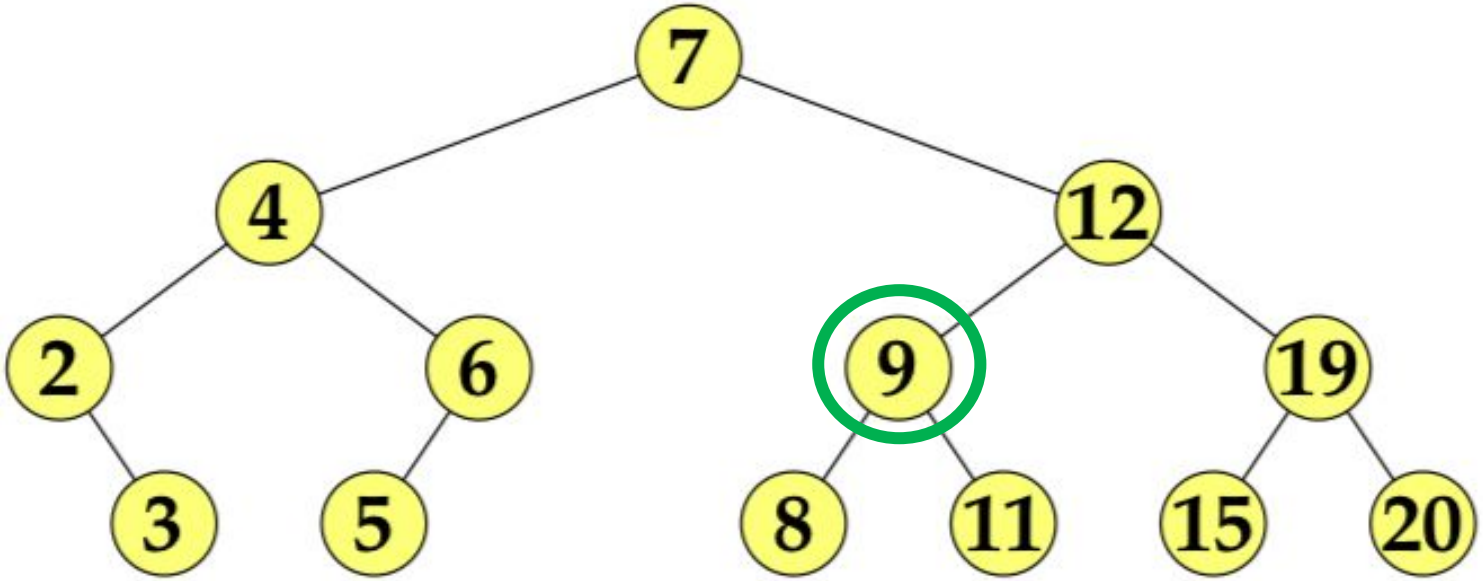


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7

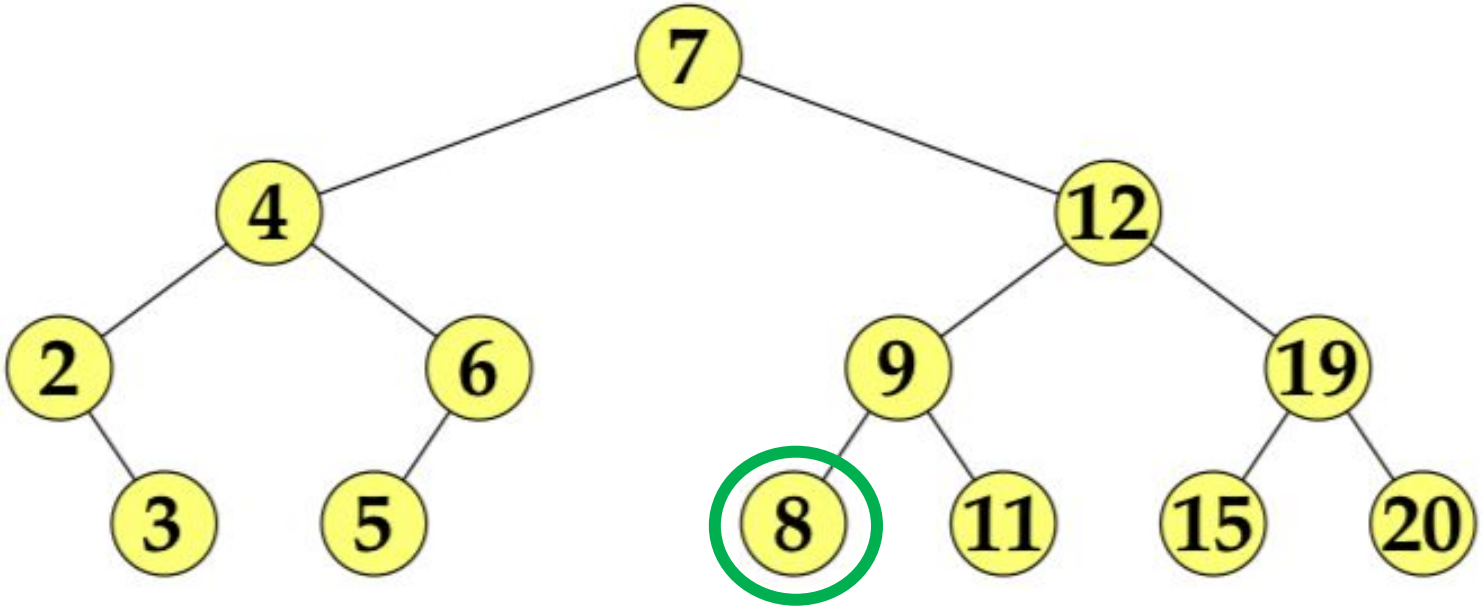


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7

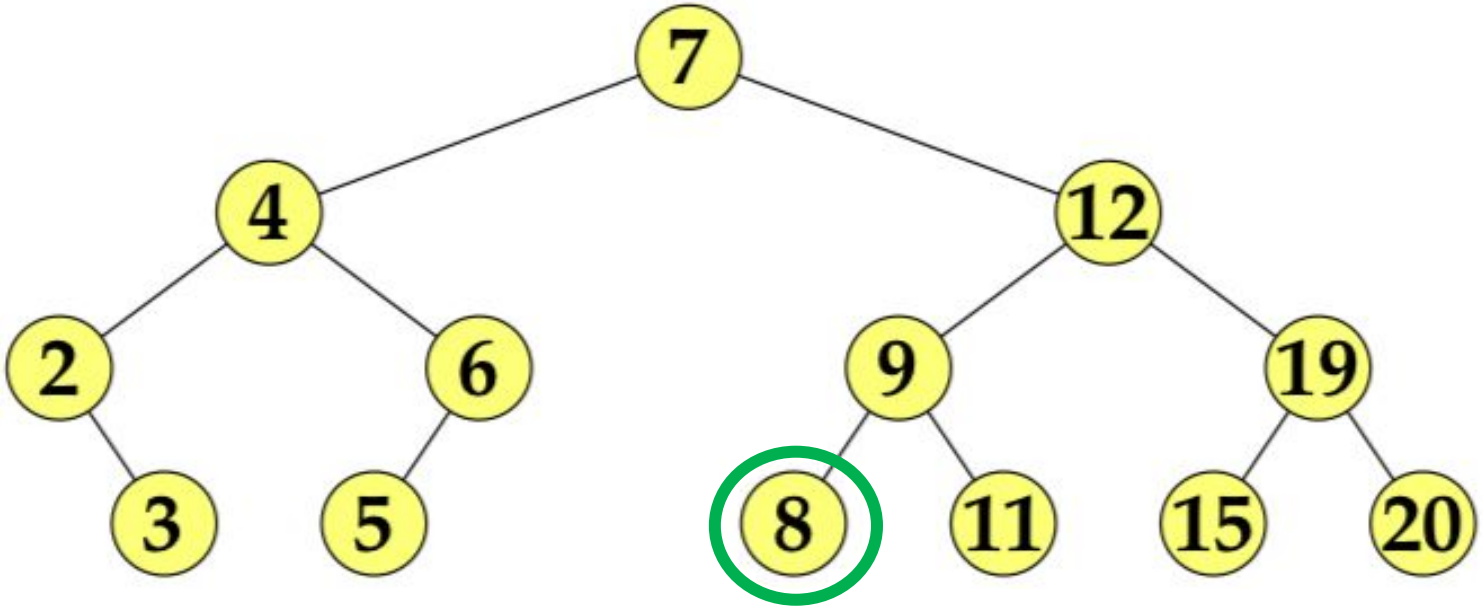


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8

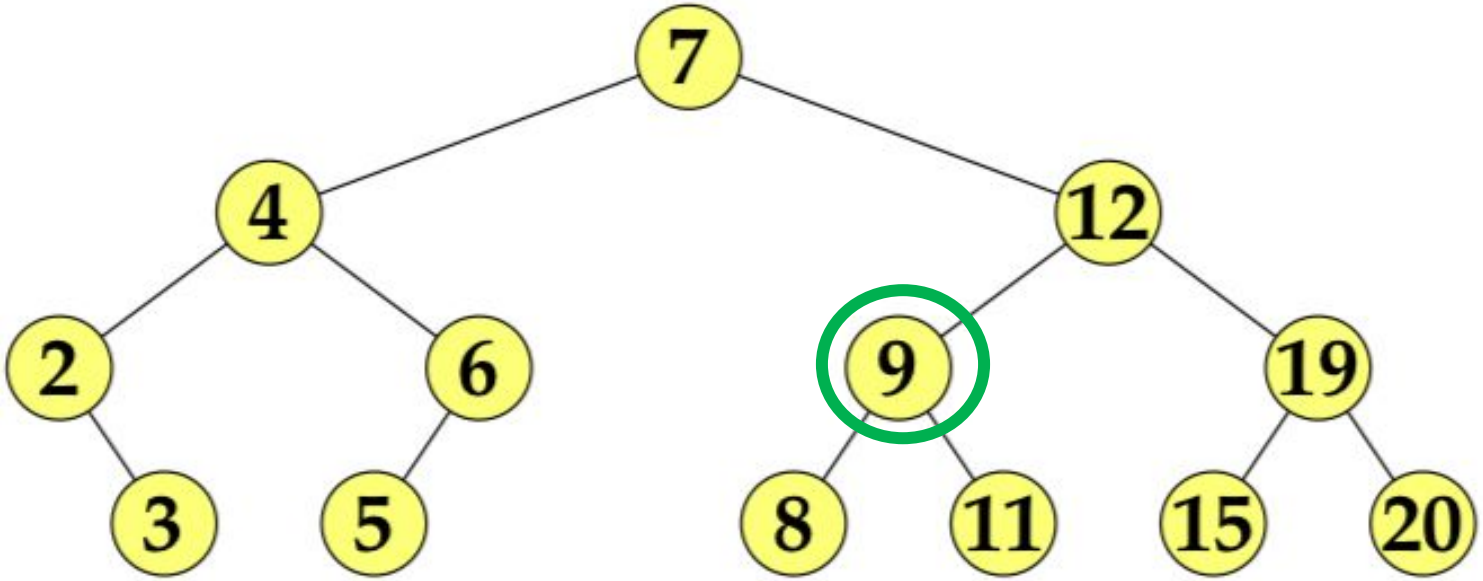


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8

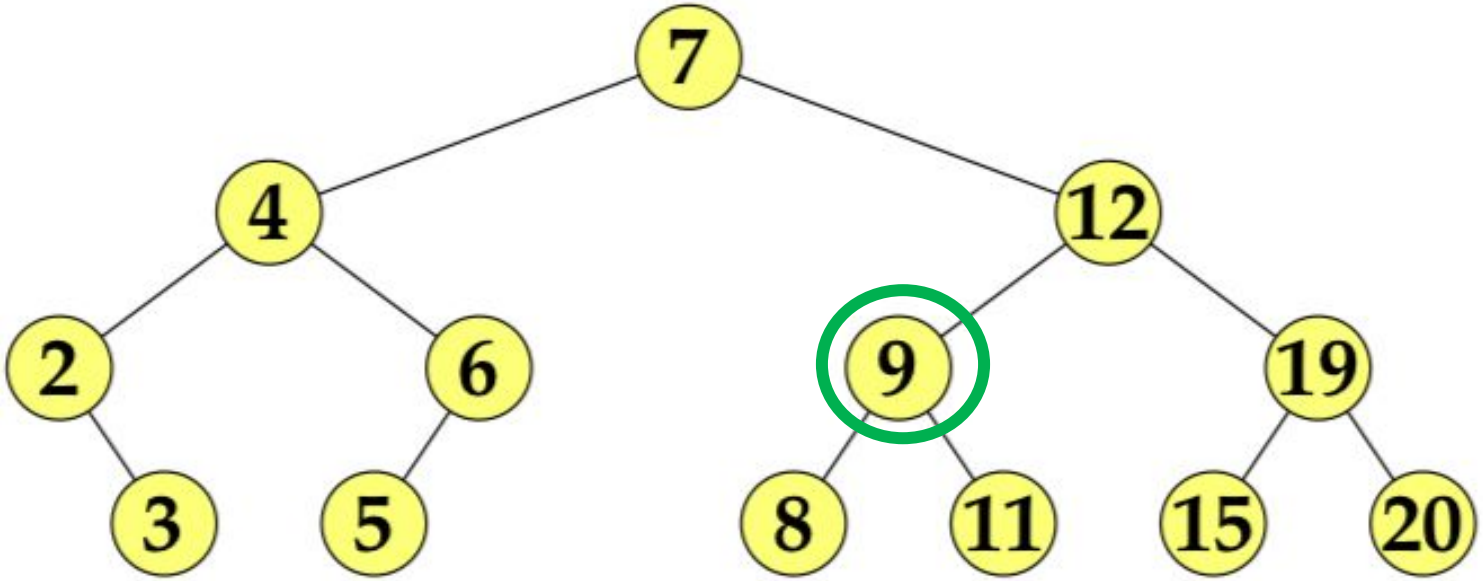


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9

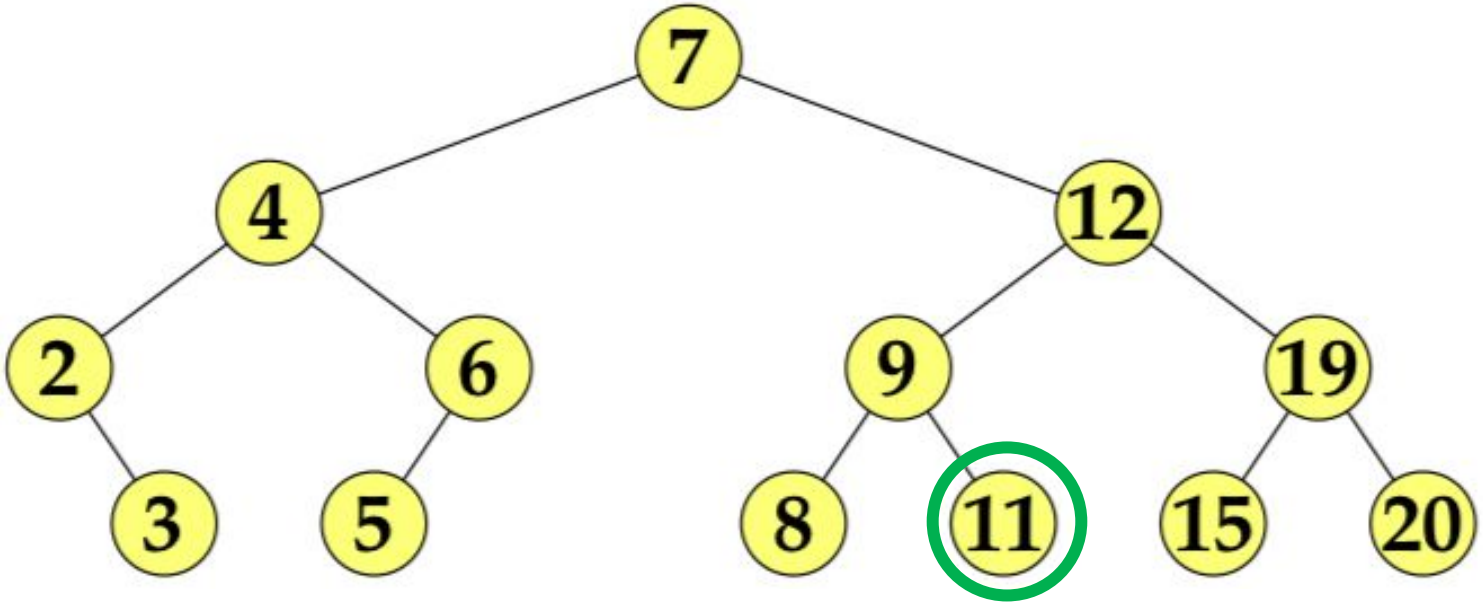


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9

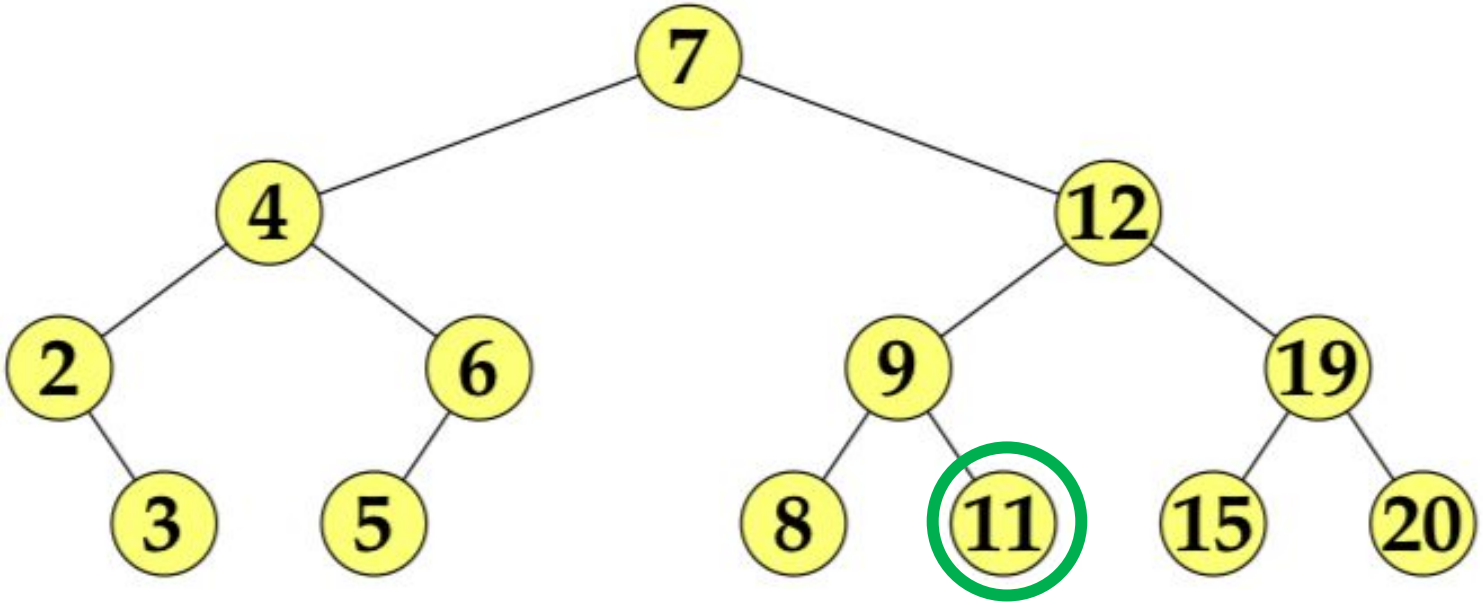


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11

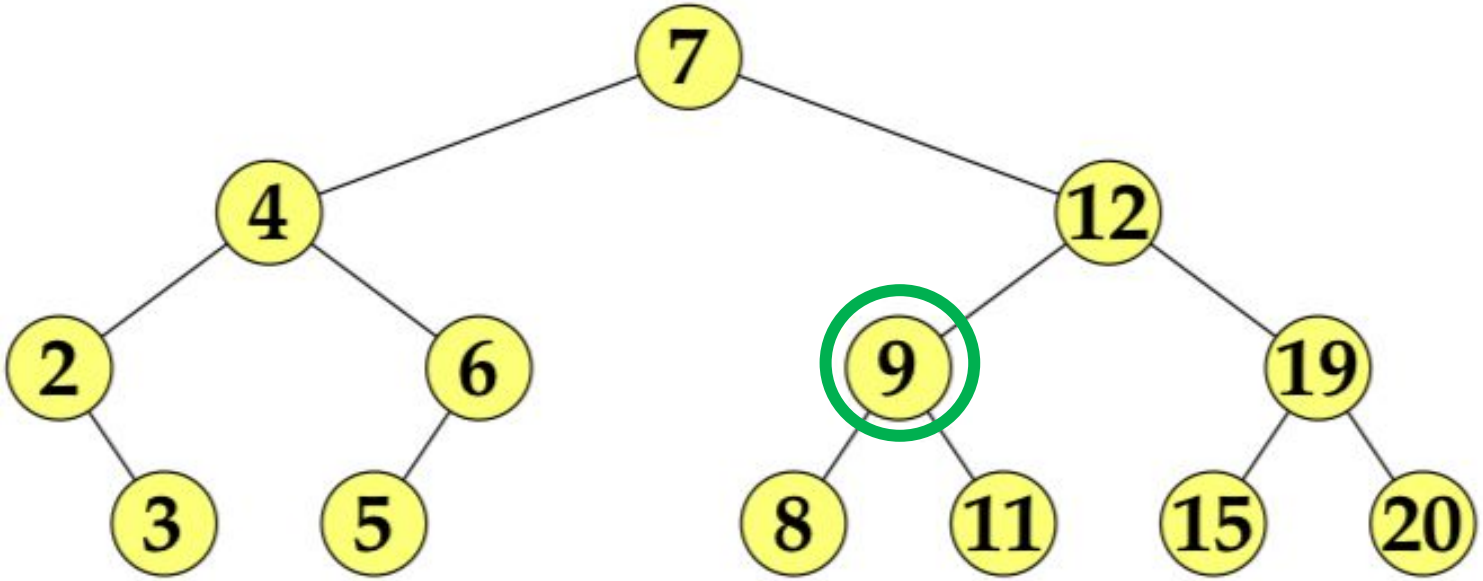


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11

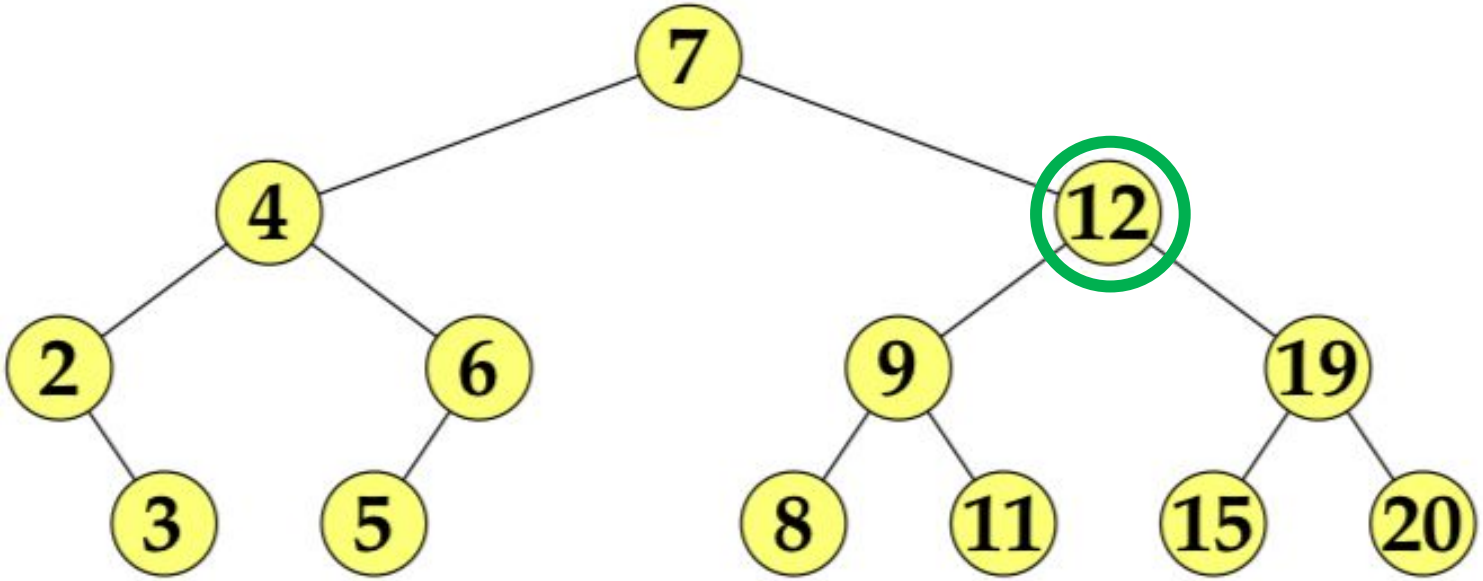


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11

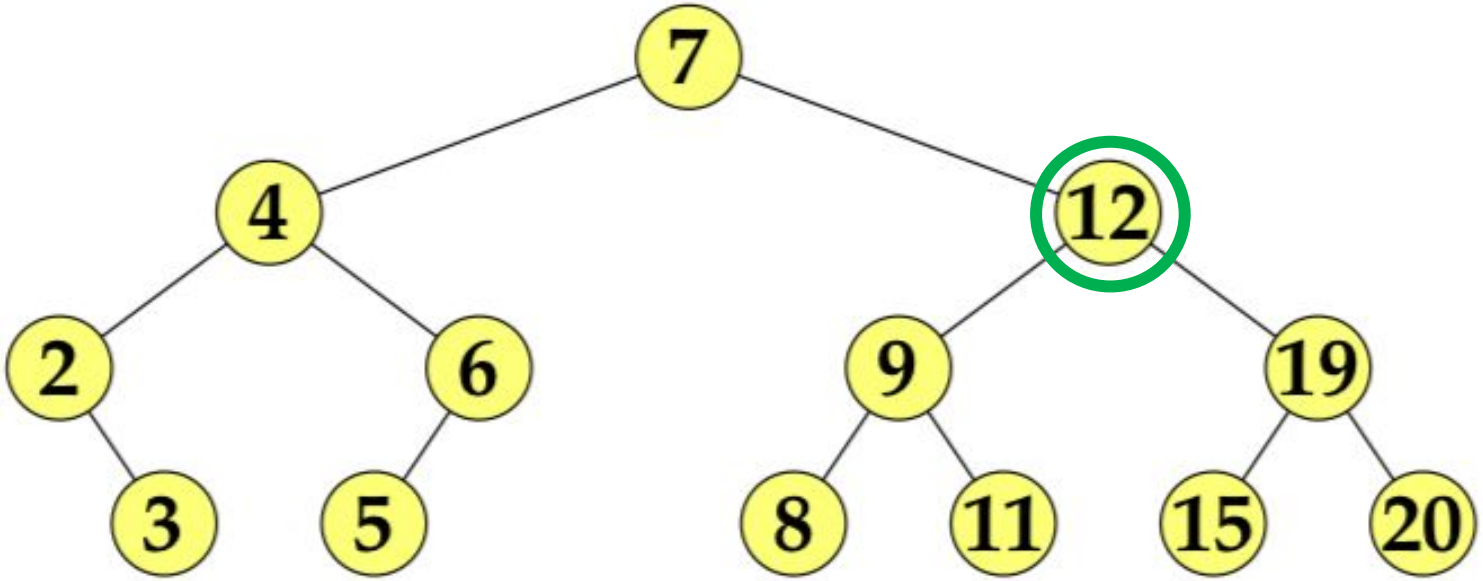


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12

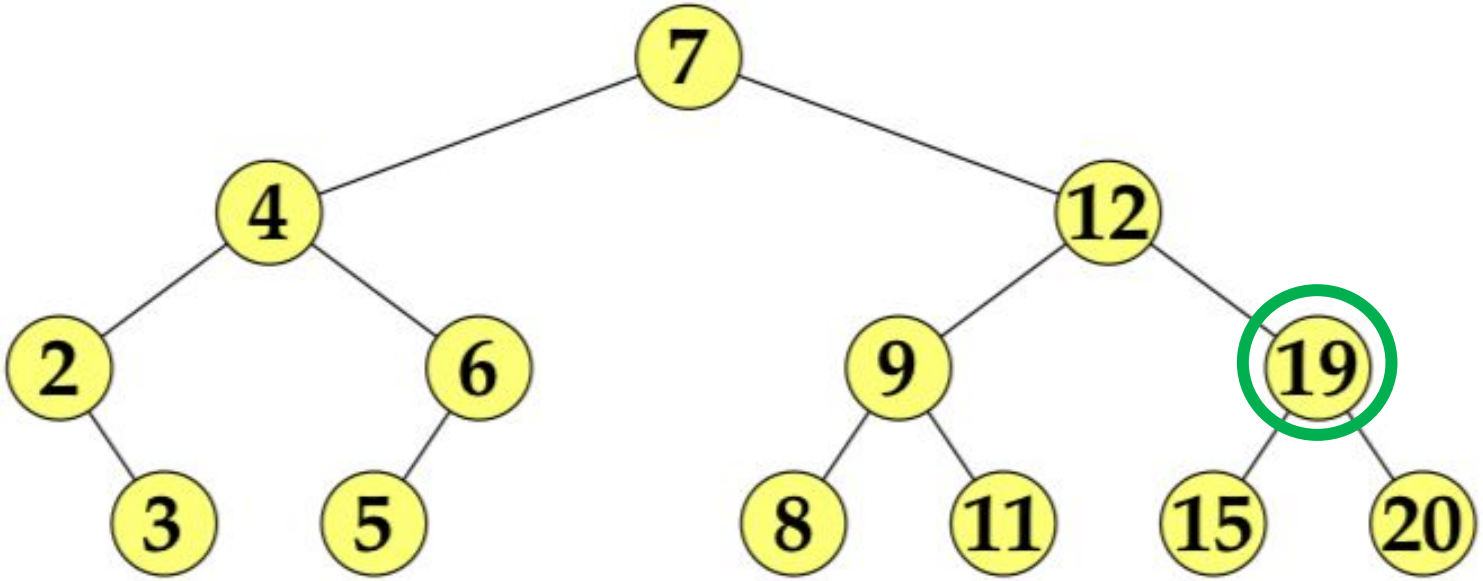


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12

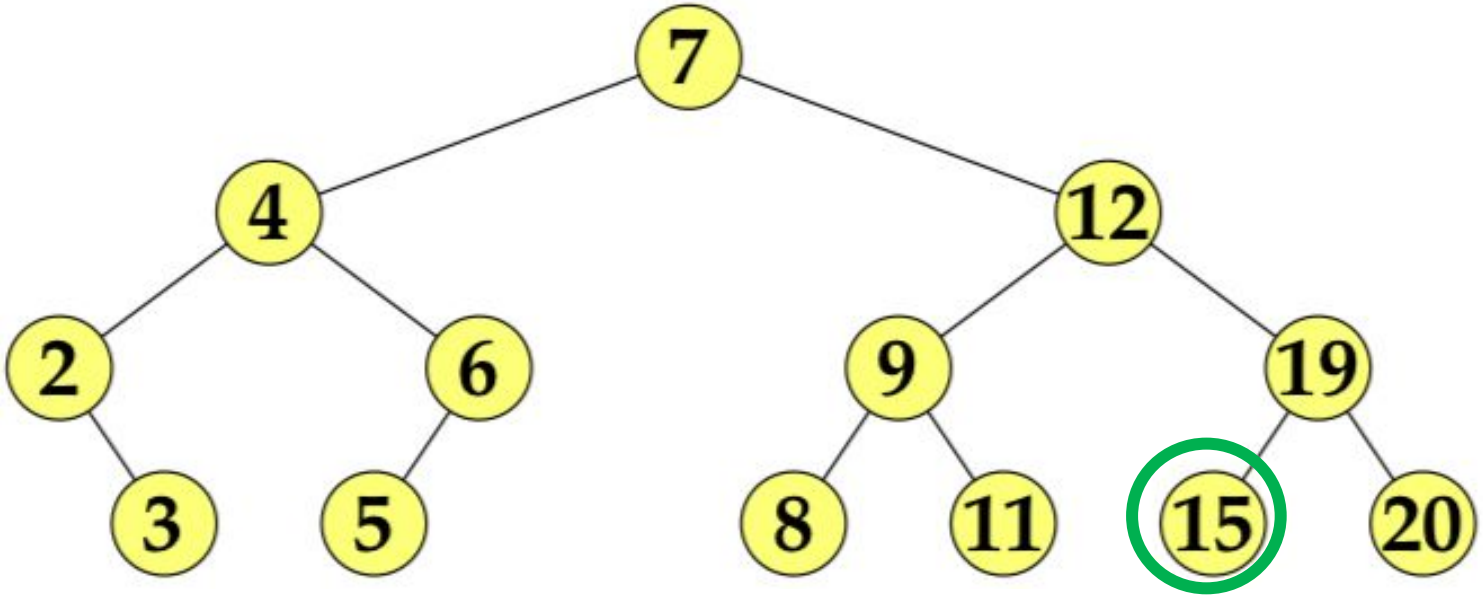


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12

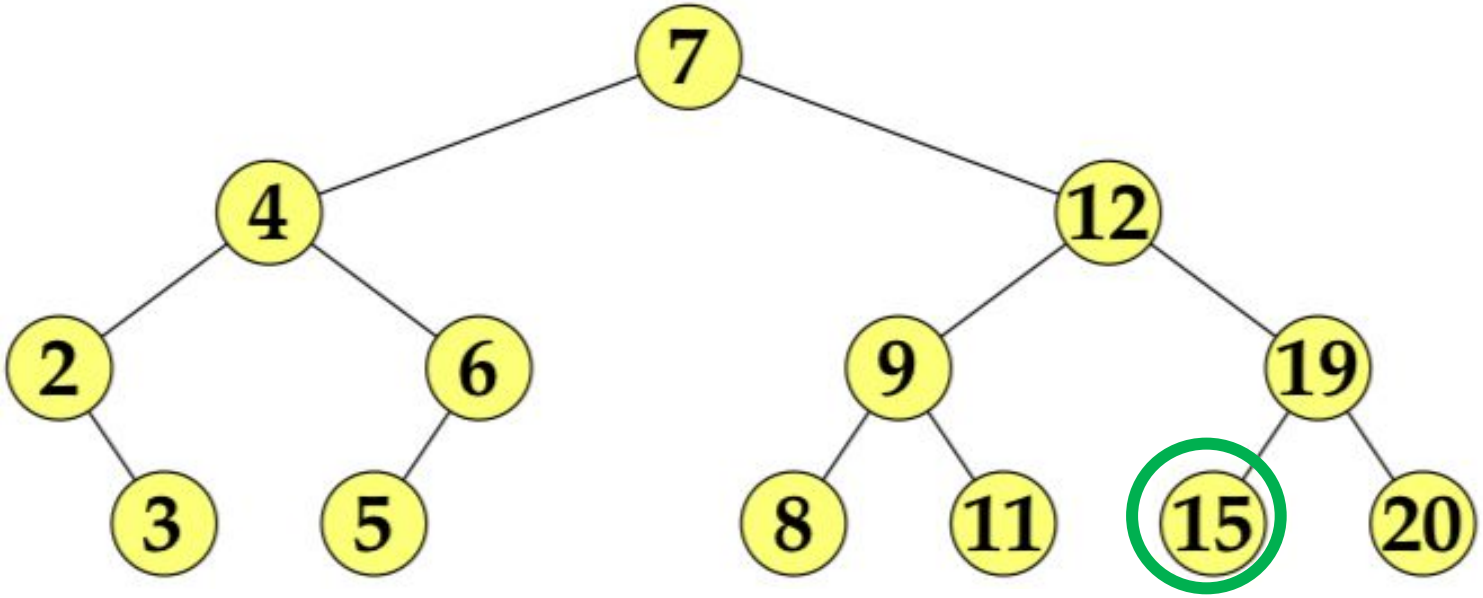


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15

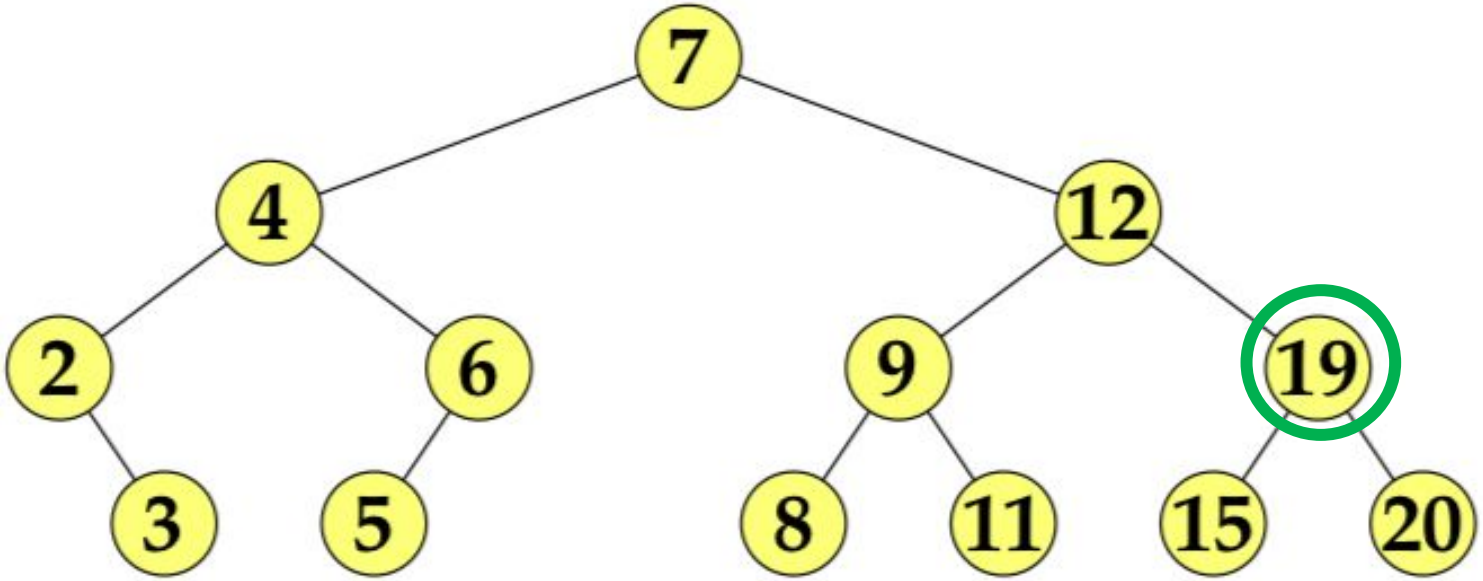


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15

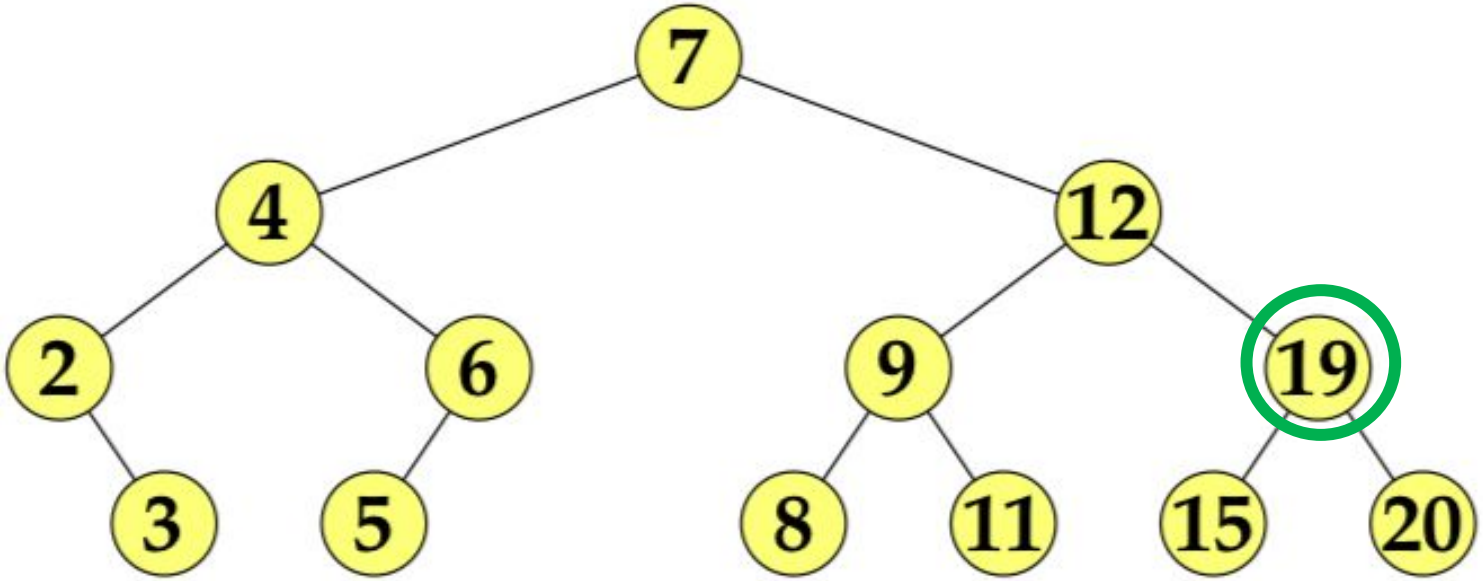


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, *19*

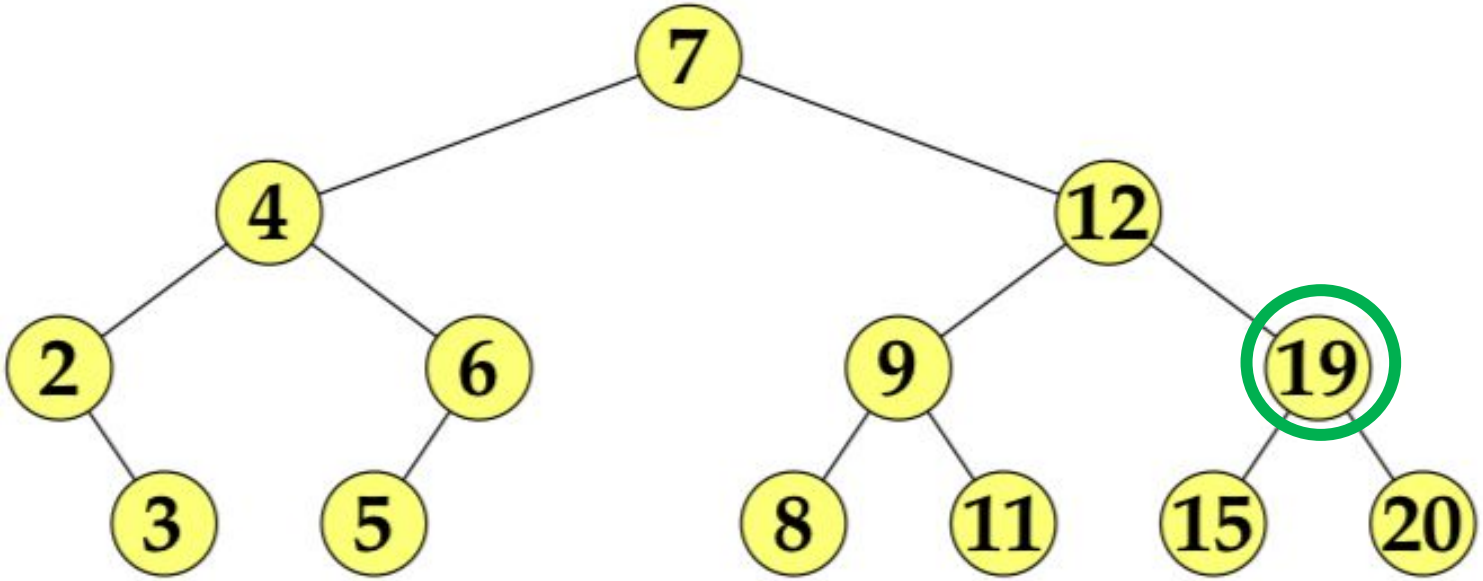


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19

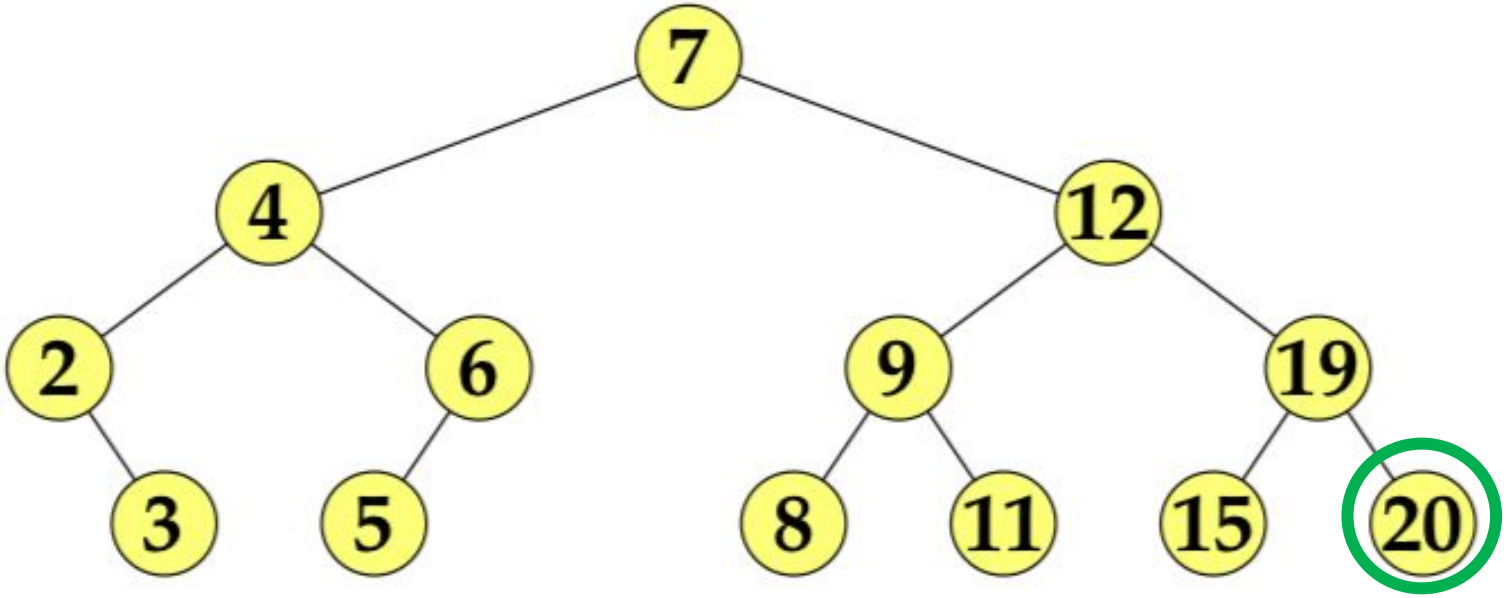


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19

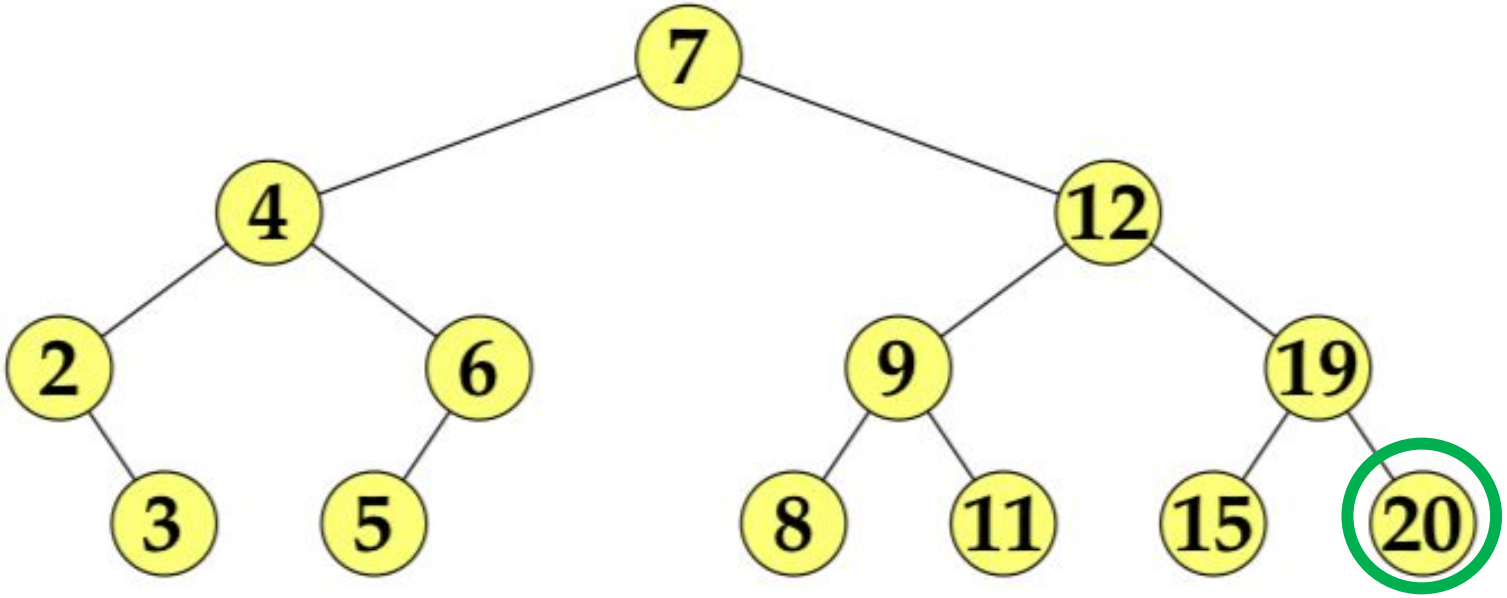


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19, 20

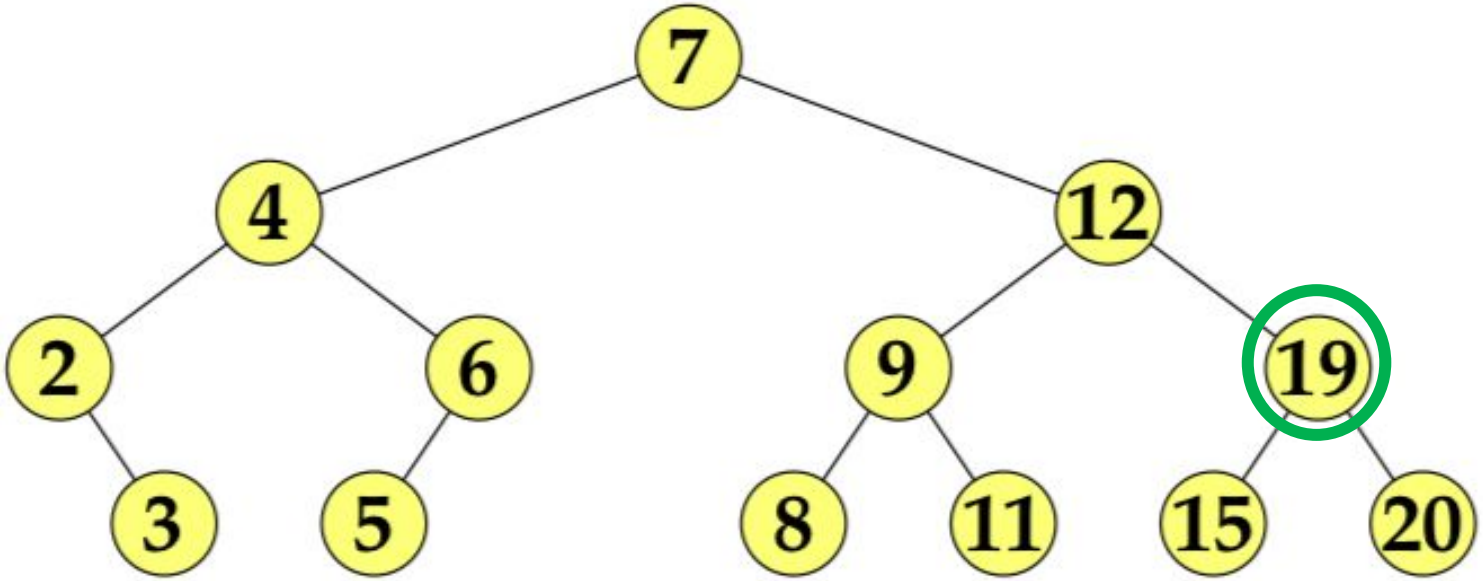


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19, 21

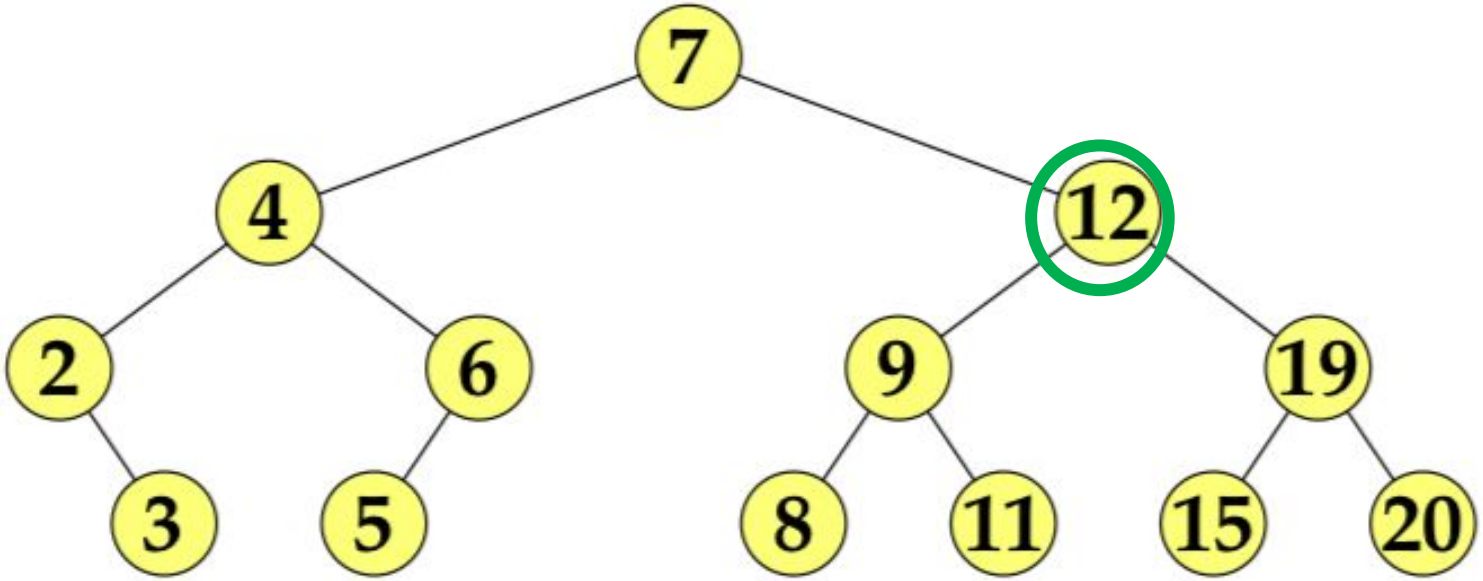


# Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19, 21

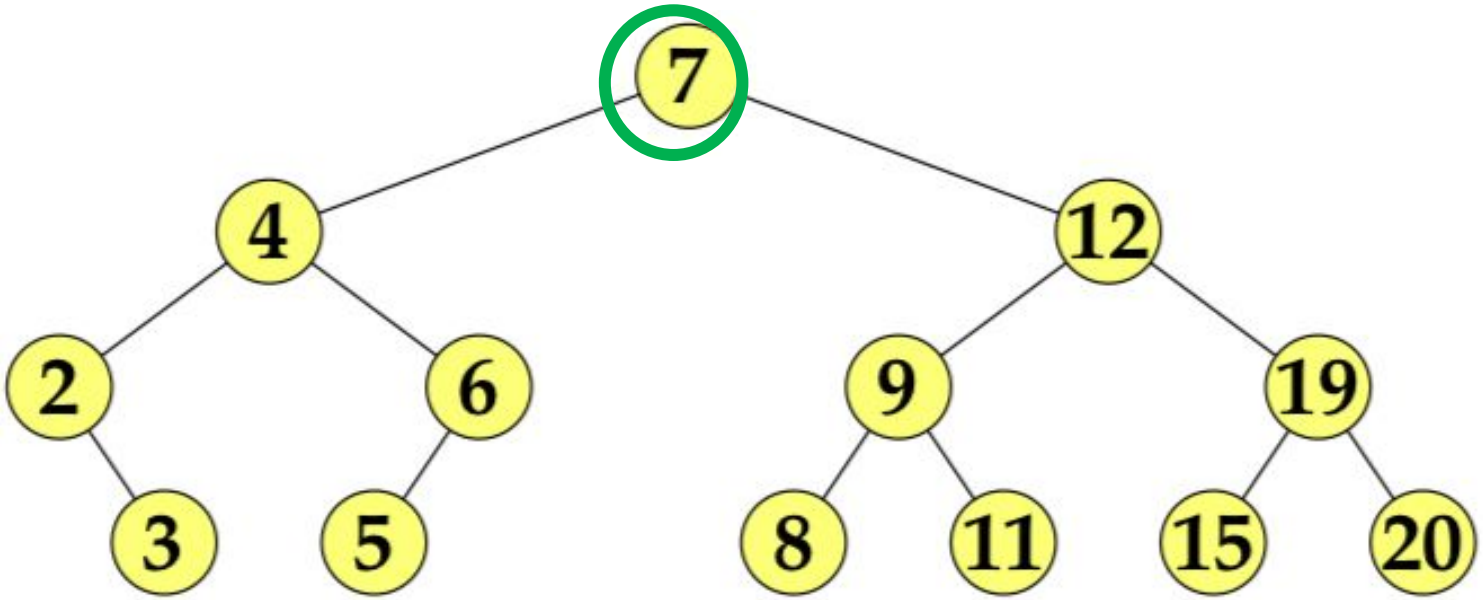


# Inorder

What would the in-order traversal be here?

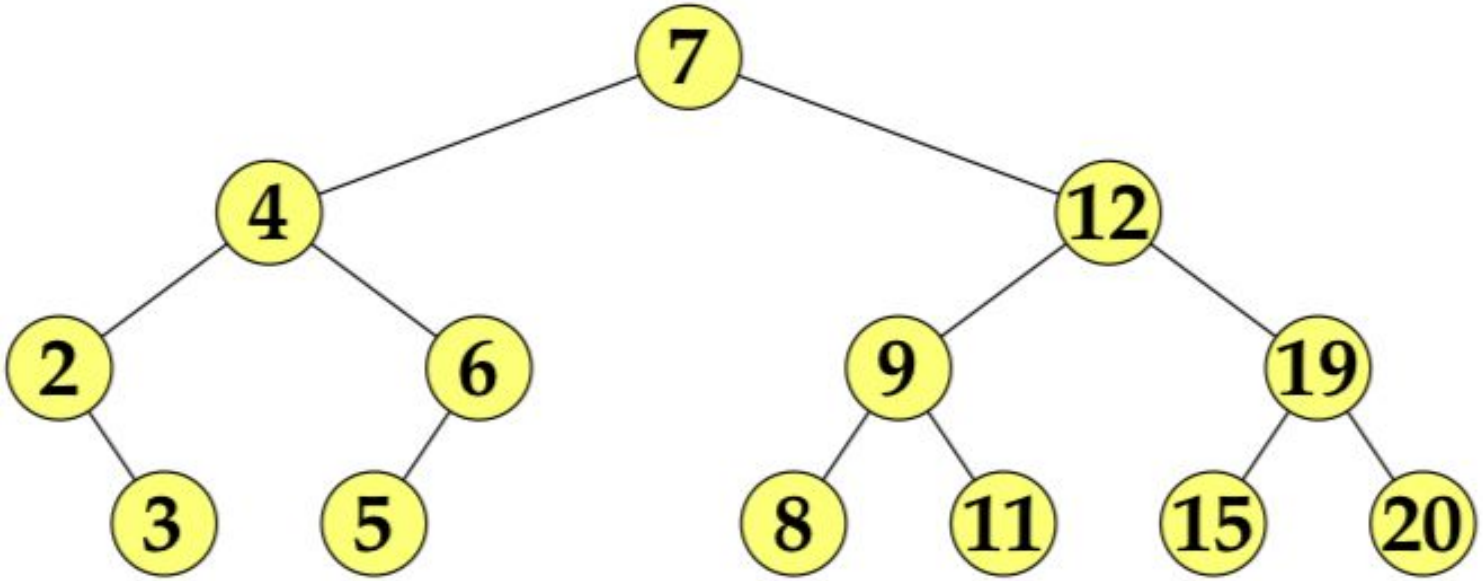
left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19, 21



# Inorder

- 2, 3, 4, 5, 6, 7, 8 , 9, 11, 12, 15, 19, 20



# Summary

**Takeaways**:

Binary search trees are an efficient data structure for search

For a *balanced* binary search tree:

- Search: O(log n)
- Insertion: O(log n)
- Removal: O(log n)